

Embedded Systems and IoT

Ingegneria e Scienze Informatiche - UNIBO

a.y. 2025/2026

Lecturer: Prof. Alessandro Ricci

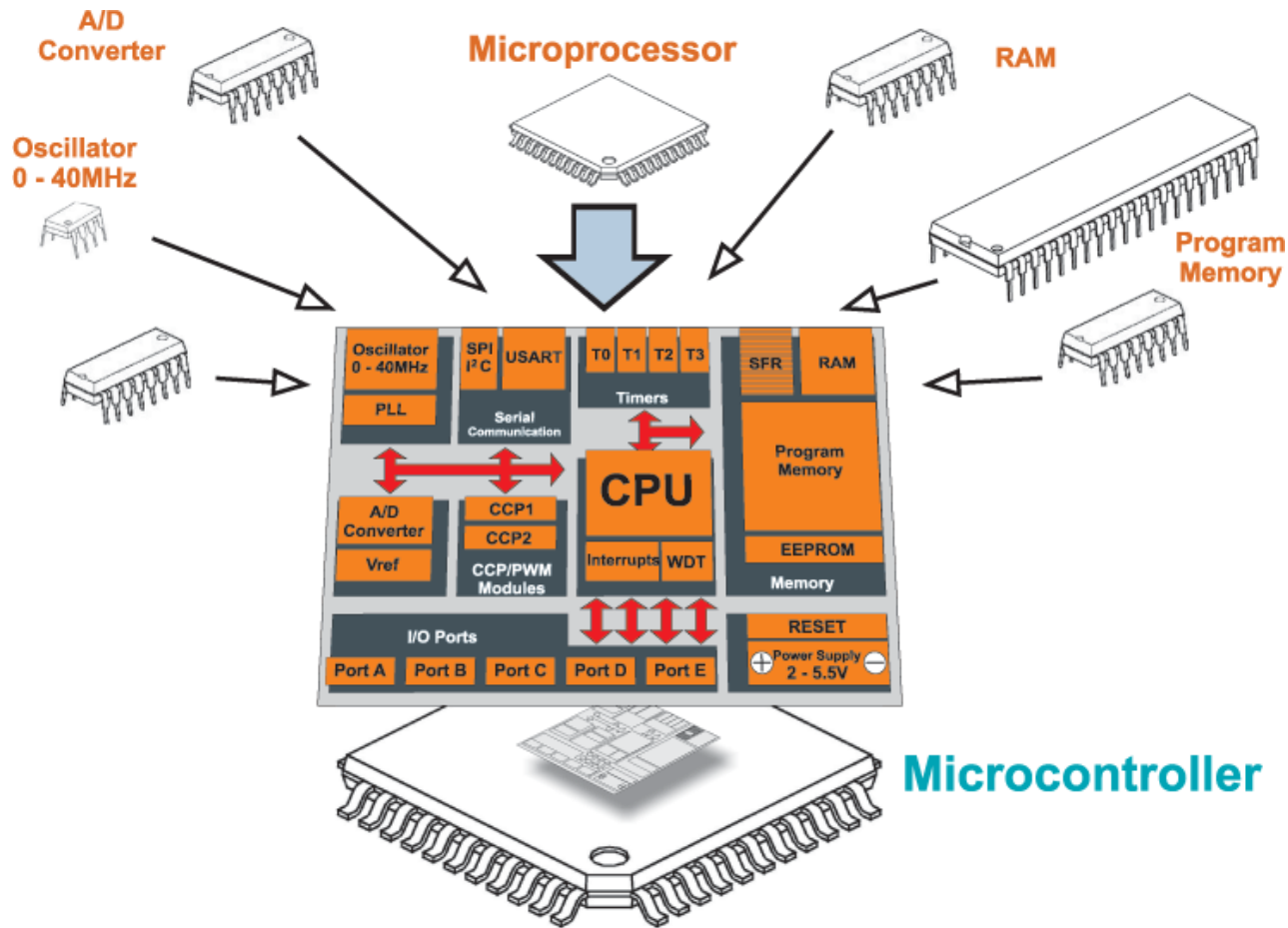
[module 1.2]

**EMBEDDED SYSTEMS BASED
ON MICROCONTROLLERS**

SUMMARY

- In this module we discuss the main component of a microcontroller and basic microcontroller programming
 - CPU and Memory
 - GPIO
 - Interrupts
 - Timer
 - Serial communication
- Arduino Uno and Wiring framework are used as reference platform and programming framework
- The concepts introduced here will be seen in practice in lab

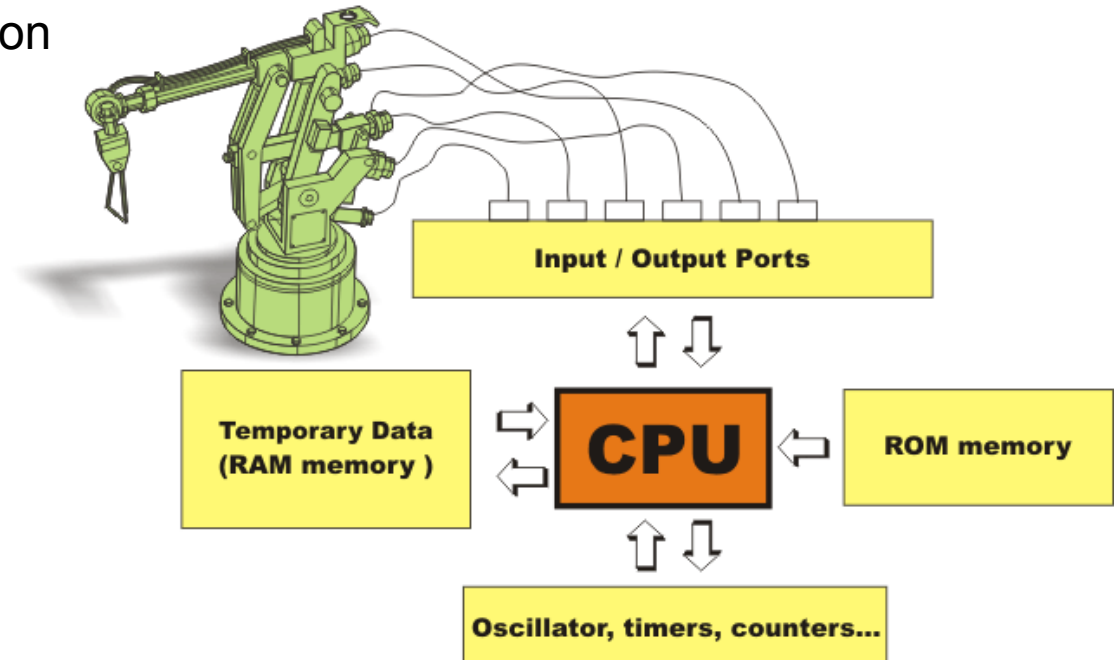
MICROCONTROLLER ELEMENTS



(image from: <http://www.mikroe.com/chapters/view/74/pic-basic-book-chapter-1-world-of-microcontrollers/>)

MICROCONTROLLER ELEMENTS

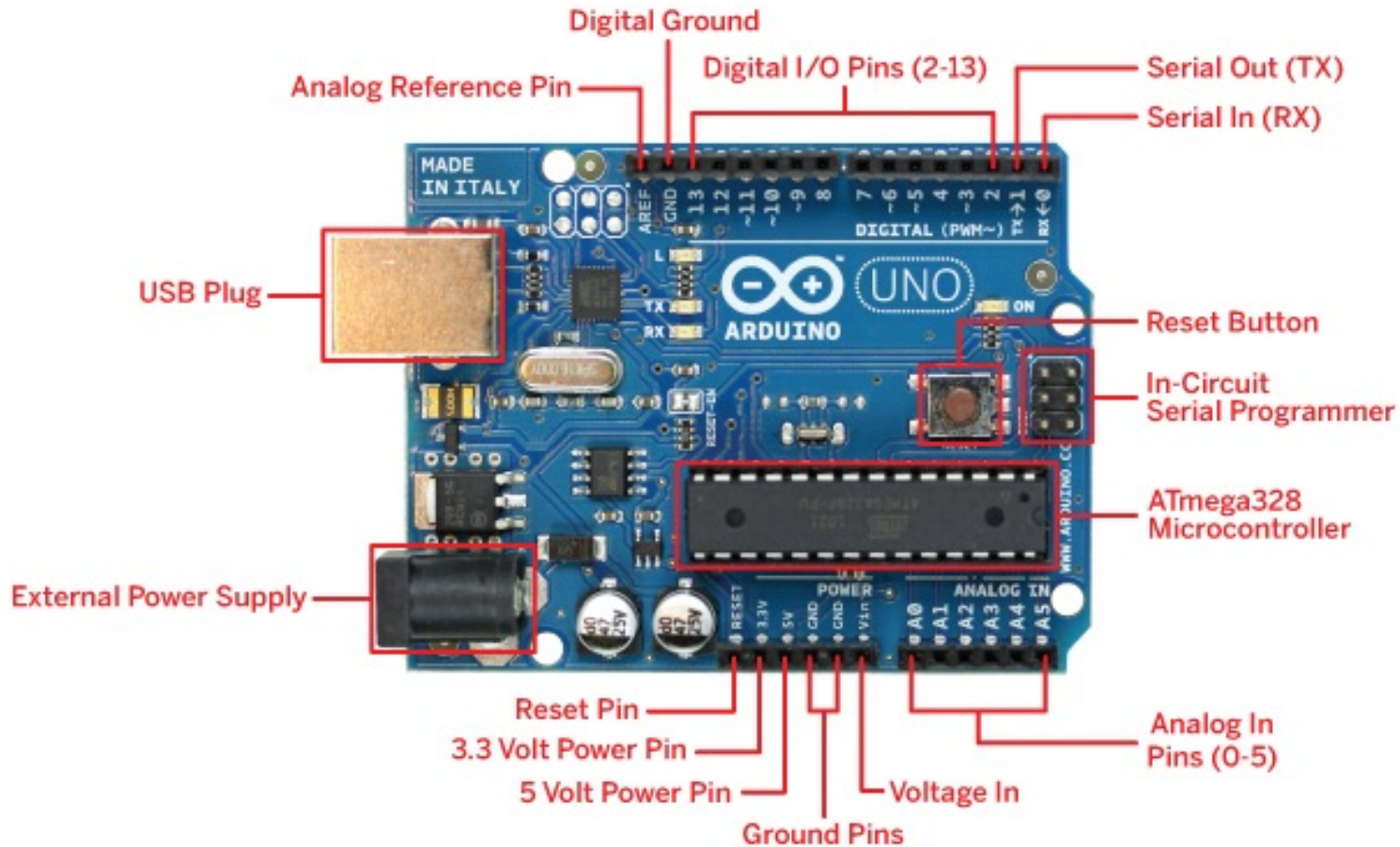
- CPU
- Memory unit
- Ports for general purpose Input/Output (GPIO)
- Analog/digital converters
- Timers
- Bus for serial communication
- Clock
- Power circuit



A REFERENCE EXAMPLE: ARDUINO UNO ARCHITECTURE

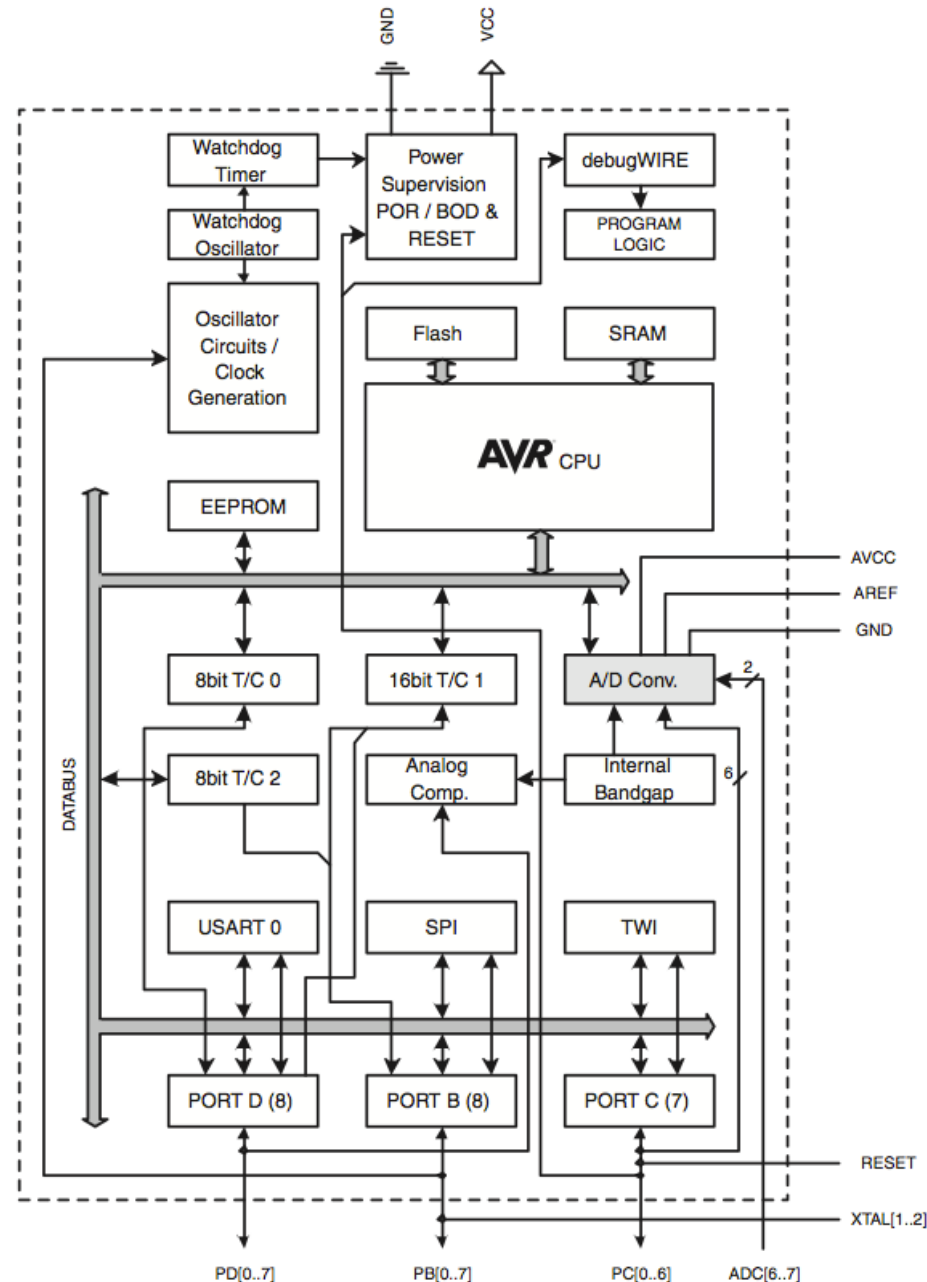
- Main components
 - **MCU ATMega328P**
 - 8 bit, 16 MHz
 - Flash memory: 32 KB
 - SRAM memory: 2 KB, EEPROM: 1 KB
 - **14 pins for digital input/output**
 - 6 can be used PWM outputs
 - **6 analog inputs**
 - USB connector
 - power jack
 - ICSP header
 - reset button
- Other specs:
 - Operating voltage: 5V
 - Input voltage (recom.): 7-12 V
 - DC current per I/O pin: 40 mA
 - DC current per 3.3V: 50mA

A REFERENCE EXAMPLE: ARDUINO UNO ARCHITECTURE



MCU ATMega328P

- AVR CPU 8 bits, 16 MHz
 - 32 registers
- memory - 16 bits
 - Flash memory: 32 KB,
 - 0.5 used by the bootloader
 - SRAM memory: 2 KB
 - EEPROM: 1 KB



MCU PROGRAMMING

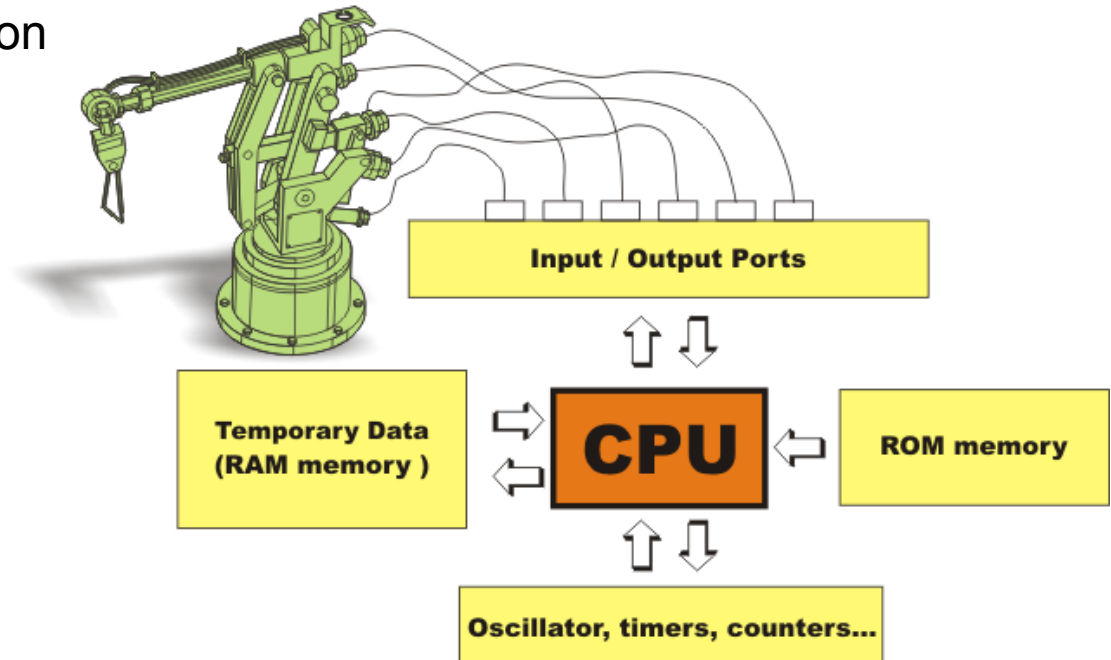
- MCU programming is done by using an external host system (e.g. a PC), where programs are edited, compiled, creating the executable (or binary)
- The executable is then uploaded to the MCU by means of specific HW, or directly by means of the serial communication with the host PC
- The MCU software stack *does not* include any operating system
 - the executable is directly copied into MCU memory and executed by the CPU

THE ARDUINO CASE

- Arduino programs are typically developed on a host PC and uploaded by means of USB cable
 - besides, an ICSP (In-Circuit Serial Programmer) interface is available for uploading the executable using a specific external programmer device
- The upload via USB is done by means of a **bootloader**
 - a small program(~0.5KiB) pre-stored (via ICSP)
- Programming environment/IDE: **Wiring** e Arduino IDE
 - Wiring is an open-source C/C++ framework, available for different kind of microcontrollers and SoC as well
 - <http://wiring.org.co/>
 - Arduino IDE is a simple but effective IDE for editing sources, compiling, uploading binaries (working not only for Arduino..)
 - based on GNU tools and other open source projects
 - **avr-gcc** and open-source libraries (e.g. AVR Libc)

MICROCONTROLLER ELEMENTS

- **CPU**
- **Memory unit**
- Ports for general purpose Input/Output (GPIO)
- Analog/digital converters
- Timers
- Bus for serial communication
- Clock
- Power circuit

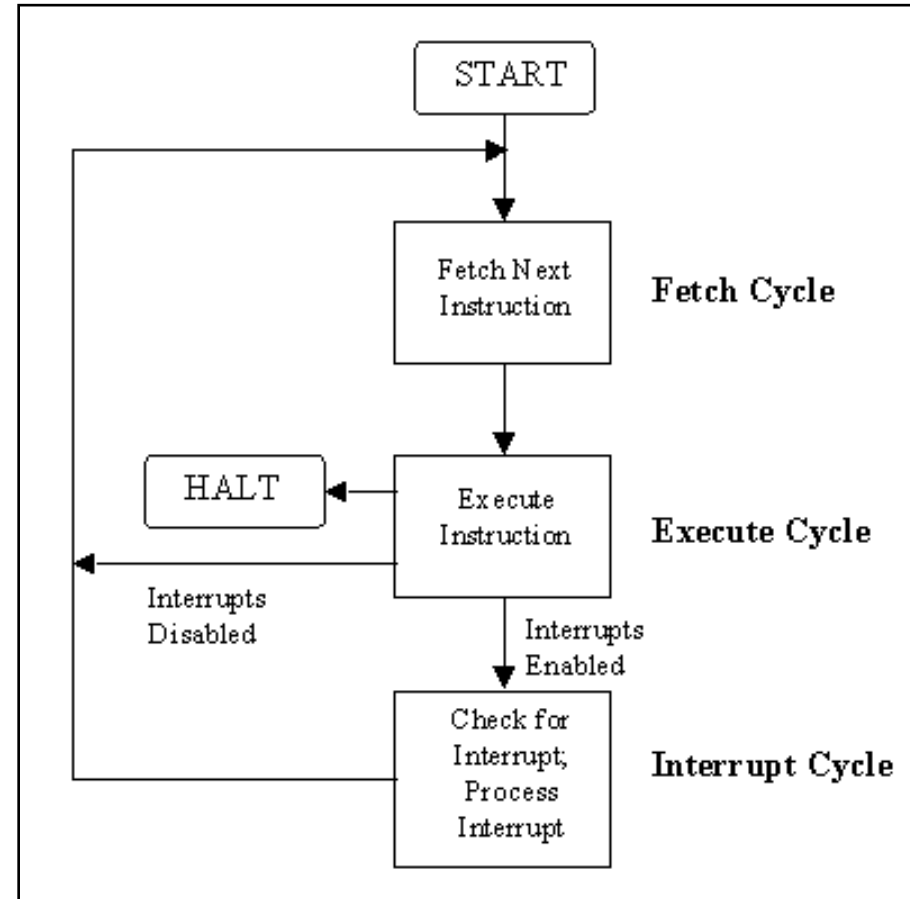


CPU ARCHITECTURE: RECALL

- The CPU behaviour follows the **Von Neumann machine model**
 - **fetch-decode-execute execution cycle**

The **instruction-execution cycle** of a computer based on Von Neumann machine consists in:

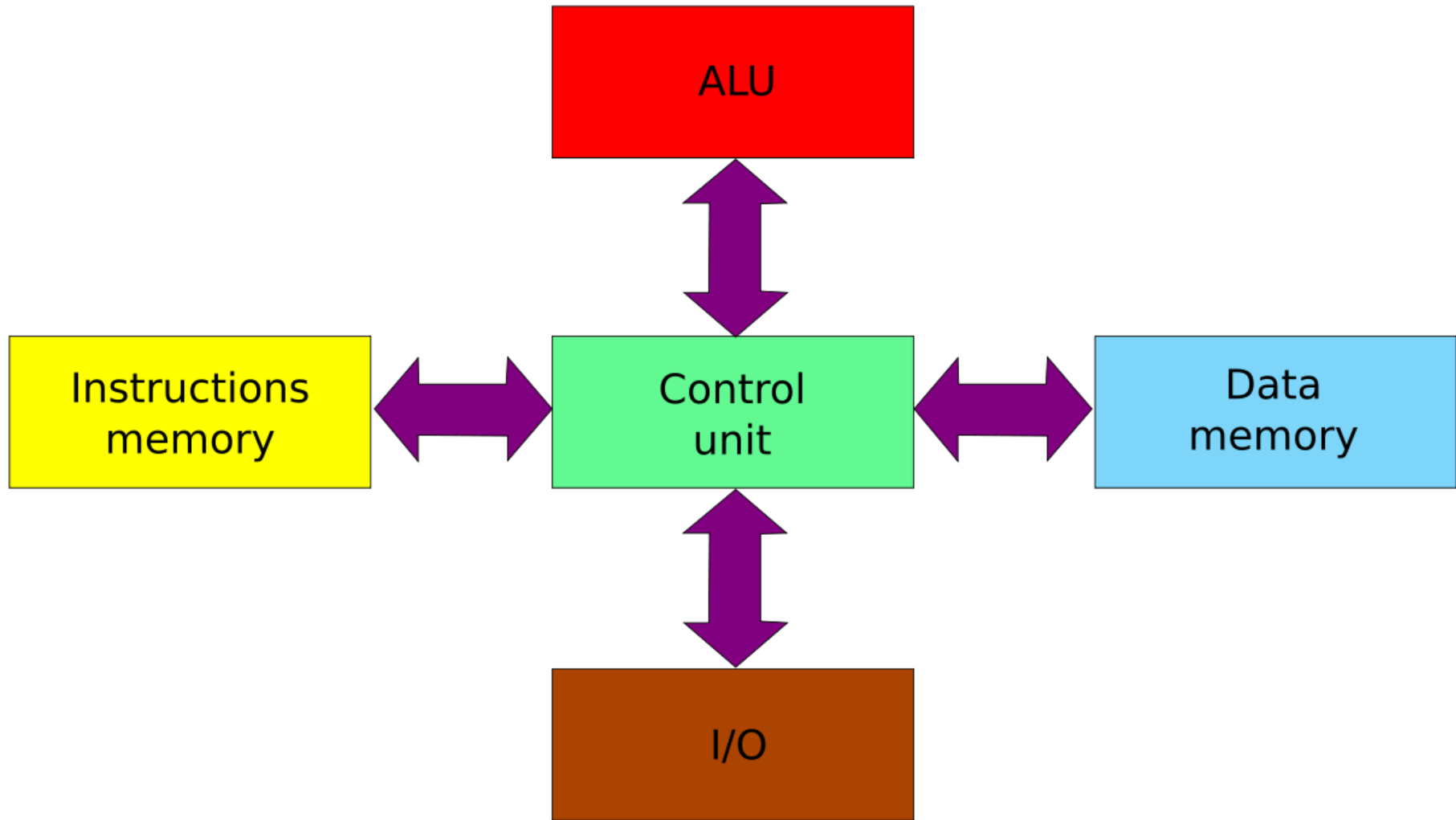
- **fetching** from the memory the next instruction to be executed, putting it in a specific register (**instruction register**)
- **decoding** of the instruction, which may lead to fetch other operands from the memory
- instruction **execution**, which may involve updating registers and writing register content to the memory



HARVARD ARCHITECTURE

- It's a variant of the Von Neumann architecture in which *instructions and data are stored in physically different/separated memories*, using distinct buses
 - the name comes from Harvard Mark I relay-based computer
 - instructions stored in punch cards and data in electro-mechanical devices
 - architecture typically used in microcontrollers and DSP, with extensions and variants
 - e.g.: Atmel AVR
- Relevant aspects:
 - the two memories have typically very different properties, for instance:
 - **code in FLASH memory**
 - fast read access, slow write access
 - **data in SRAM**
 - fast read & write access, bigger power consumption & cost

HARVARD ARCHITECTURE



EXAMPLE: MCU ATMega328P

- MCU based on **Harvard** architecture
 - code instruction
 - stored in **FLASH (non volatile)** - 32 KiB
 - 0.5 KB used by the bootloader
 - data
 - variables / stack stored in **SRAM** (volatile) - **2 KiB**
 - long-term / persistent data stored in **EEPROM** (non volatile) - **1 KiB**
- **Remarks:**
 - **the amount of memory for data/variables is very small**
=> disciplined use of data structure (e.g. minimising the use of strings)
 - the stack is stored in RAM
 - small memory => constraint on the level of call nesting and recursion

A COMPARISON AMONG MEMORIES

- **Flash** memory
 - non-volatile memory
 - fast read, slow write
 - max ~100000 write cycle
 - low power consumption and cost
- **SRAM** memory
 - volatile
 - fast read, fast write
 - on ATMega328P: ~2 clock cycles
 - bigger power consumption than FLASH
- **EEPROM** memory
 - non volatile memory
 - slow access
 - on Arduino: can be read or written using the library:
<http://arduino.cc/en/Reference/EEPROM>

CPU ARCHITECTURE

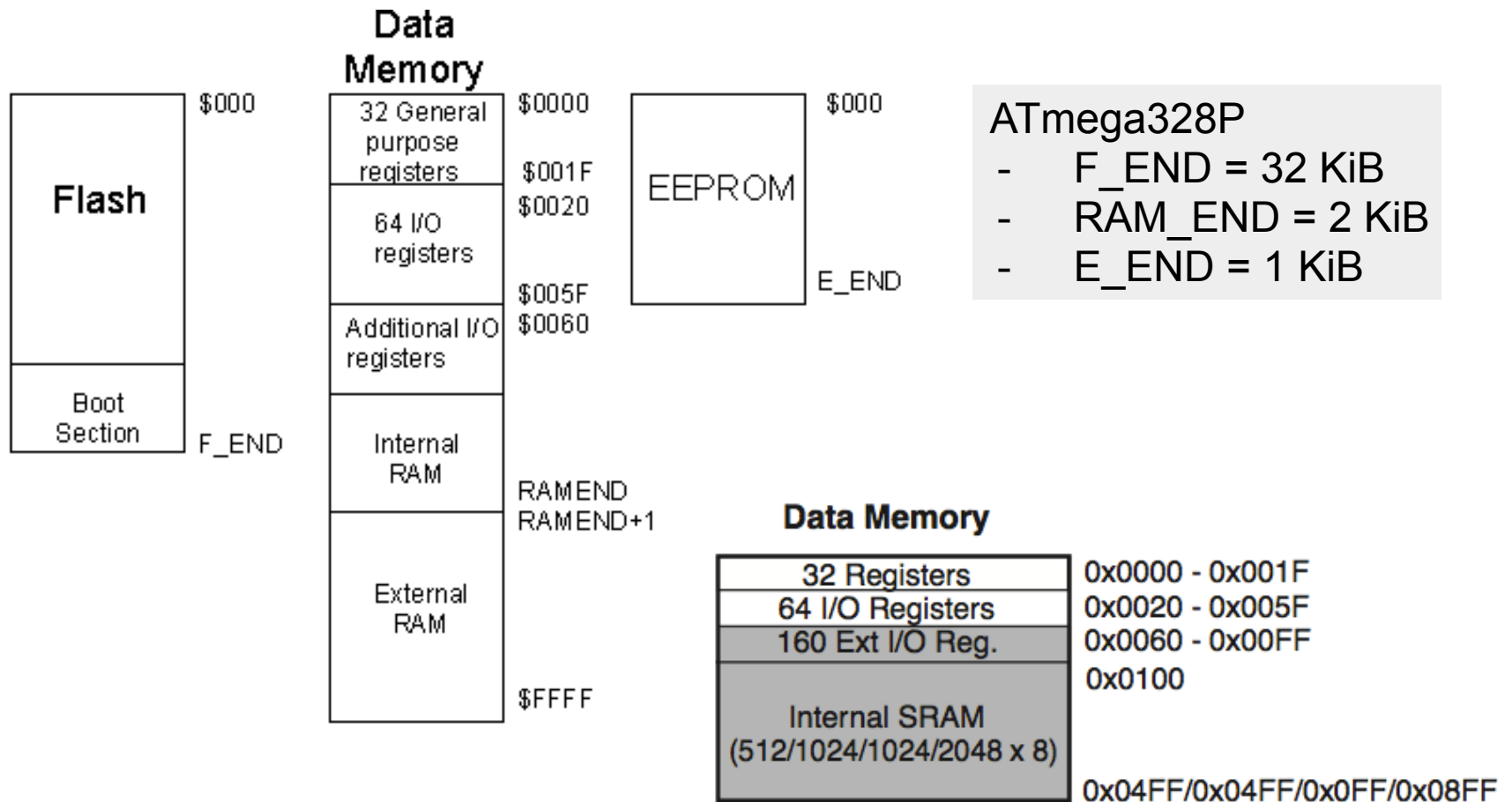
- The CPU executes program instructions encoded in machine language
- Characterised by its own **Instruction Set Architecture (ISA)**
 - set of possible instructions recognised by the CPU
 - set of registers
 - general or programmable registers (GPR)
 - e.g. accumulator
 - special purpose registers (SFR)
 - Program counter (PC)
 - » storing the address of the next instruction to be executed
 - Program Status Word (PS)
 - » information about processor state
 - XX bit (XX = 8, 16, 32, 64) CPU
 - number of bits used in registers, to specify memory addresses and values

EXAMPLE: AVR8 ARCHITECTURE

- AVR8 is the microprocessor inside the ATmega328P microcontroller
 - RISC architecture - code: 8 bits, data: 16 bits - 16 MHz
- CPU registers
 - **32 general-purpose 8 bits registers (R0–R31)**
 - all arithmetic and boolean operations involve these registers
 - only load and store operations access the RAM memory
 - small set of special-purpose registers:
 - **PC**: 16- or 22-bit program counter
 - **SP**: 8- or 16-bit stack pointer
 - **SREG**: 8-bit status register - traffic light:
 - C Carry flag (used in the subtraction)
 - Z Zero flag - set when an arithmetic result is zero
 - I Interrupt flag - set when interrupts are enabled

MEMORY ADDRESSING

- Both general-purpose CPU registers and I/O registers (described in the following) are mapped into low memory (and accessible using memory addresses in that range)



A LOOK AT AVR ASSEMBLY LANGUAGE

- AVR Instruction set - categories:
 - **arithmetic**
 - **bit manipulation**
 - **memory/registers transfer**
 - **jump**
 - **test/branch**
 - **call**
- Look at
 - https://en.wikipedia.org/wiki/Atmel_AVR_instruction_set
 - full manual:
 - <http://www.atmel.com/images/Atmel-0856-AVR-Instruction-Set-Manual.pdf>

ABOUT PERFORMANCE

- Microprocessor clock frequency: 16 MHz
 - period: $0.0625\text{ us (microsec)} = 62.5\text{ ns (nanosec)}$
- The most part of the instructions are executed using 1 o 2 clock cycles, i.e. $\sim 60\text{ns}$, $\sim 125\text{ns}$
 - precise and detailed data are available on CPU datasheet
- This information allows to make accurate predictions about the execution time of programs or parts of them, in particular in the worst case
 - important aspect for real-time systems

BASIC CONTROL ARCHITECTURE:

SUPER LOOP

- The super-loop is the simplest control architecture (a framework) adopted for microcontroller programming
 - very common and widespread since it does not require any specific HW support (such as interrupts)
- Features
 - initialisation
 - infinite loop, repeatedly executing some task

```
/* Main.c */

#include "X.h"

void main(void){
    X_Init(); // Prepare for task X
    while (1) // for ever 'super loop'
    {
        X(); Perform the task
    }
}
```

```
/* X.h - header */

void X_Init();
void X(void);
```

```
/* X.c - implementation */

void X_Init() {...}
void X(void) {...}
```

SUPER-LOOP FEATURES

- **Pro**
 - simplicity
 - the framework does not require any OS to work
 - portability
 - reliability and safety
 - efficiency
- **Cons**
 - timing not accurate
 - fragility and low flexibility
 - the basic version: 'full-power' => high power consumption

SUPER-LOOP IN WIRING

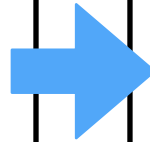
- Framework **wiring** (<http://wiring.org.co/>)
 - open-source framework for programming microcontrollers
 - reference language: C/C++
 - subset of standard ANSI C++
 - **super-loop** control architecture
 - **setup()** procedure e **loop()** procedure
 - must be implemented by the programmer
 - there is a hidden main calling these procedures
 - Wiring provides also a set of basic API
 - classes, procedures e functions in C/C++
- Source compilation and executable creation by means of a C/C++ compile + linker
 - **avr-gcc** (GNU)
- Among the other tools included in the Arduino distribution: avr-objdump
 - to disassemble an executable

FROM WIRING TO THE EXECUTABLE

- Compiling an empty program in Wiring
 - from C source code to AVR8 machine language

```
void setup(){  
}  
  
void loop(){  
}
```

empty program in
Wiring



```
00000090 <setup>:  
 90:      08 95          ret  
  
00000092 <loop>:  
 92:      08 95          ret  
  
00000094 <initVariant>:  
 94:      08 95          ret  
  
00000096 <main>:  
 96:      0e 94 a4 00    call 0x148      ; 0x148 <init>  
 9a:      0e 94 4a 00    call 0x94       ; 0x94 <initVariant>  
 9e:      0e 94 48 00    call 0x90       ; 0x90 <setup>  
 a2:      c0 e0          ldi r28, 0x00 ; 0  
 a4:      d0 e0          ldi r29, 0x00 ; 0  
 a6:      0e 94 49 00    call 0x92       ; 0x92 <loop>  
 aa:      20 97          sbiw r28, 0x00 ; 0  
 ac:      e1 f3          breq .-8        ; 0xa6 <main+0x10>  
 ae:      0e 94 00 00    call 0          ; 0x0 <__vectors>  
 b2:      f9 cf          rjmp .-14       ; 0xa6 <main+0x10>  
 ...
```

Executable disassembly
(created by the tool avr-objdump, -d option)

CONTROL LOOP DISASSEMBLY

```
00000090 <setup>:
  90:  08 95                ret
00000092 <loop>:
  92:  08 95                ret
...
00000096 <main>:
int main(void)
{
    init();
  96:  0e 94 a4 00          call    0x148      ; 0x148 <init>
    initVariant();
  9a:  0e 94 4a 00          call    0x94       ; 0x94 <initVariant>
    setup();
  9e:  0e 94 48 00          call    0x90       ; 0x90 <setup>
    for (;;) {
        loop();
a2:  c0 e0                ldi     r28, 0x00      ; 0
a4:  d0 e0                ldi     r29, 0x00      ; 0
a6:  0e 94 49 00          call    0x92       ; 0x92 <loop>
        if (serialEventRun) serialEventRun();
aa:  20 97                sbiw    r28, 0x00      ; 0
ac:  e1 f3                breq    .-8           ; 0xa6 <main+0x10>
ae:  0e 94 00 00          call    0           ; 0x0 <__vectors>
b2:  f9 cf                rjmp    .-14          ; 0xa6 <main+0x10>
```

THE EXAMPLE

```
int c;
```

```
void setup(){
  c = 100;
}
```

```
void loop(){
  c = c + 1;
}
```

...

000000a6 <setup>:

a6:84 e6	ldi	r24, 0x64	; 100
a8:90 e0	ldi	r25, 0x00	; 0
aa:90 93 01 01	sts	0x0101, r25	
ae:80 93 00 01	sts	0x0100, r24	
b2:08 95	ret		

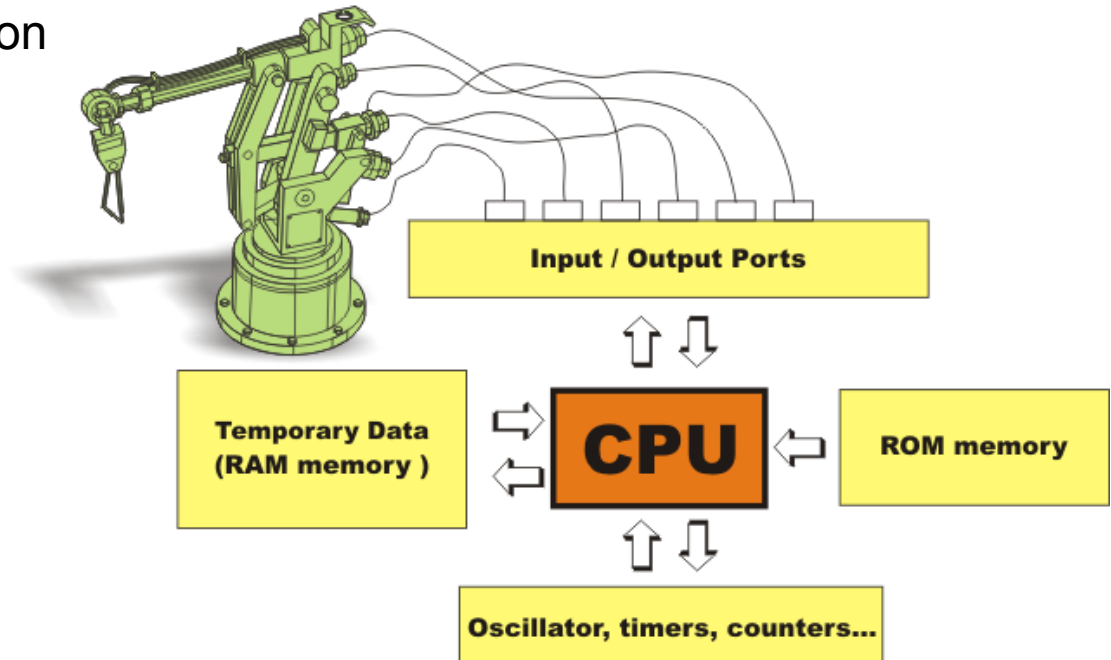
000000b4 <loop>:

b4:80 91 00 01	lds	r24, 0x0100	
b8:90 91 01 01	lds	r25, 0x0101	
bc:01 96	adiw	r24, 0x01	; 1
be:90 93 01 01	sts	0x0101, r25	
c2:80 93 00 01	sts	0x0100, r24	
c6:08 95	ret		

...

MICROCONTROLLER ELEMENTS

- CPU
- Memory unit
- **Ports for general purpose Input/Output (GPIO)**
- **Analog/digital converters**
- Timers
- Bus for serial communication
- Clock
- Power circuit



GENERAL-PURPOSE I/O (GPIO)

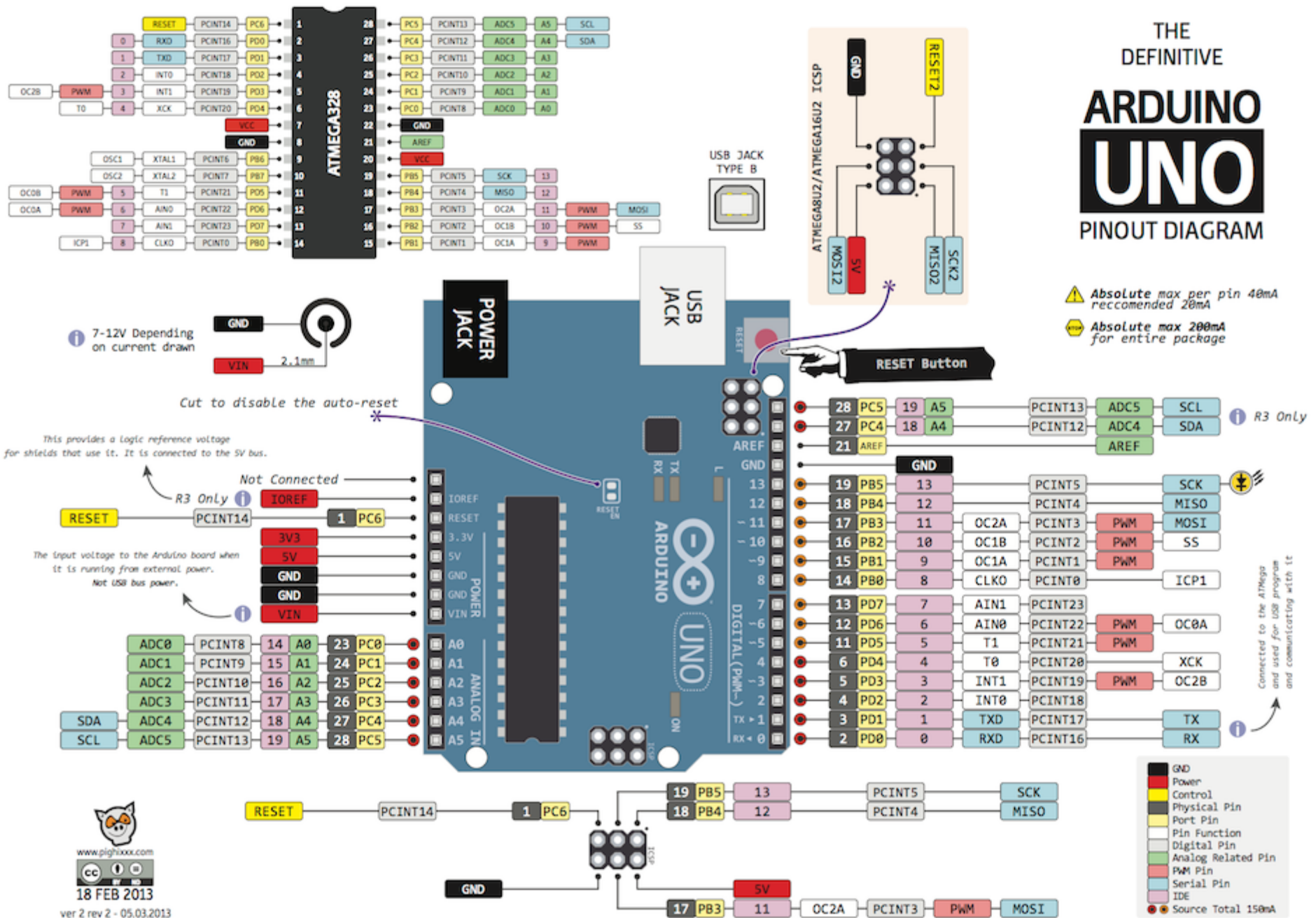
- Every microcontroller typically features a set of **pin** che can be directly used to drive input/output
- Pins typically are *general-purpose*: can be programmed to work as **input** or **output** depending on the need
 - input
 - the value can be read by program
 - output
 - the value is controlled by the program
- Pins can be either **analog** or **digital**
 - a digital pin can assume two values only, HIGH or LOW (1 o 0)
 - an analog pin can assume any value in a range (0...5volts)

EXAMPLE: ARDUINO GPIO

- Arduino pins
 - **14 digital pins**
 - configurable either as input or output
 - **6 analog pins**
 - input only
- Some pins are multiplexed, i.e. serving multiple functions
 - e.g.: pin 0 e pin 1 are also used as serial port (for transmitting and receiving bits)

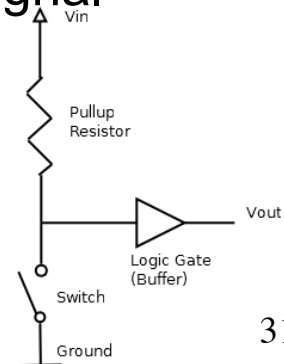
THE DEFINITIVE ARDUINO UNO PINOUT DIAGRAM

- ⚠ Absolute max per pin 40mA recommended 20mA
- ⚠ Absolute max 200mA for entire package



OPERATING PARAMETERS

- Reference operating parameters for pins:
 - **voltage** (in Volt)
 - to be applied on the pin (input case) or produced by the pin (output case)
 - example: 5 V (Arduino), 3.3 V (Raspberry Pi and ESP)
 - **current** (in Ampère)
 - that can receive (input) or that can be produced (output)
 - example: Arduino: 40 mA, ESP8266 12mA
- Either presence or absence of *pull-up circuits/resistors*
 - internally, the pins can be equipped with *pull-up circuits*, that allow for fix the voltage at the max value (referred as +VCC) even when the pin — configured as input — is not connected to any circuit, i.e. it is in open-circuit state, to avoid malfunctioning due to signal floating
 - resistors of tens of KOhm are used for this purpose
 - this point will be clarified in lab slides

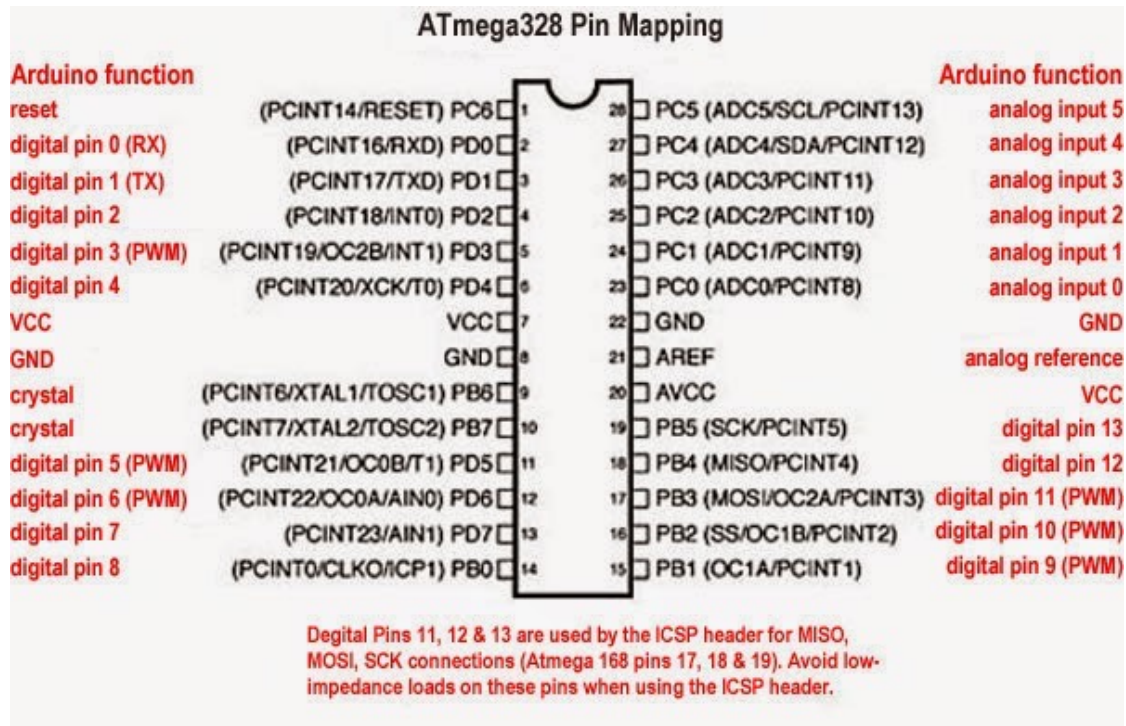


INPUT/OUTPUT PORTS

- The MCU interacts with the pins (and then with external devices) by means of **ports**
 - as for the pins, a port can be configured to be either input or output (to receive signals or to send signals)
- A port is managed by one or multiple **special purpose registers (SRF)** connected to the pin that carry the digital or analog signal, sent or received by the external device
 - keeping the state (current value) + current configuration (input/output)

EXAMPLE: ATmega328P

- L'ATmega328P has 23 I/O lines grouped by **3 8-bit ports** named **B, C** and **D**
 - port D manages pins 0..7, port B pins 8..13, and port C the analog pins
- Each pin is identified by a label such as PD0, PC1 or PB2.
 - for instance, PD0 identifies pin 0 of the port D



ATMega328P - PORTS AND REGISTERS

- Each port is managed by 3 registers: DDR_x, PORT_x, PIN_D_x, where x is the port name. For instance, for the port D;
 - DDR_D
 - this register is both Read/Write and stores the configured **direction** for the pins (bit set to 0 => pin configured as INPUT; bit set to 1 => pin configured as OUTPUT)
 - PORT_D
 - this register is both Read/Write and stores the pin **state**, which changes depending on the pin configuration:
 - for INPUT pins, then a bit set to 1 activates the PULL-UP resistor, while a bit set to 0 deactivates it;
 - for OUTPUT pins, then a bit set to 1 represents a state set to HIGH on the corresponding pin, while a bit set to 0 represents a state set to LOW
 - PIN_D
 - this register is read-only, and stores, for INPUT pins, the signal value read connected to the pin: 1 if the signal value is HIGH; 0 if the signal value is LOW.

ATMega328P - PORTS AND REGISTERS

- Therefore, to configure an OUTPUT pin to the HIGH value, one must:
 - write on the DDRx register to set the direction (OUTPUT)
 - write on the PORTx register to set the state (HIGH)
- As already mentioned, in the ATmega328 MCU, and in many other microcontrollers and general-purpose microprocessors as well, I/O registers are mapped into the memory
 - so that they can be uniformly accessed/manipulated but writing/reading bytes/words at the specific memory addresses
 - in the ATMega328P, they are located starting from the address 0x20, just after the general purpose registers

SETTING, WRITING, READING A PIN IN WIRING

- Wiring API to work with pins:
 - **pinMode()**
 - to set the pin direction (INPUT/OUTPUT)
 - **digitalWrite()**
 - to set the value of an OUTPUT pin
 - **digitalRead()**
 - to read the value of an INPUT pin
- This API internally works with port registers as discussed before

OUTPUT EXAMPLE: BLINKING LED

```
#define LED_PIN 13

void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    digitalWrite(LED_PIN, HIGH);
    delay(1000);
    digitalWrite(LED_PIN, LOW);
    delay(1000);
}
```

CODE DETAIL: delay

- **delay()** is a procedure available in Wiring that executes a *busy waiting* for the specified amount of time in milliseconds:

```
void delay(unsigned long ms)
{
    unsigned long start = millis();

    while (millis() - start <= ms)
        ;
}
```

from wiring.c
source code
(open-source)

- This procedure calls another procedure: **millis()**
 - it returns the current number of milliseconds elapsed since the power on of the device
 - it accesses variables managed by a timer (discussed later in this module)
- **IMPORTANT REMARK**
 - *delay() and busy waiting blocks the loop, impacting on reactivity of the system*

INPUT/OUTPUT EXAMPLE: BUTTON-LED

```
const int buttonPin = 2;    // the number of the pushbutton pin
const int ledPin = 13;     // the number of the LED pin

// variables will change:
int buttonState = 0;       // variable for reading the pushbutton status

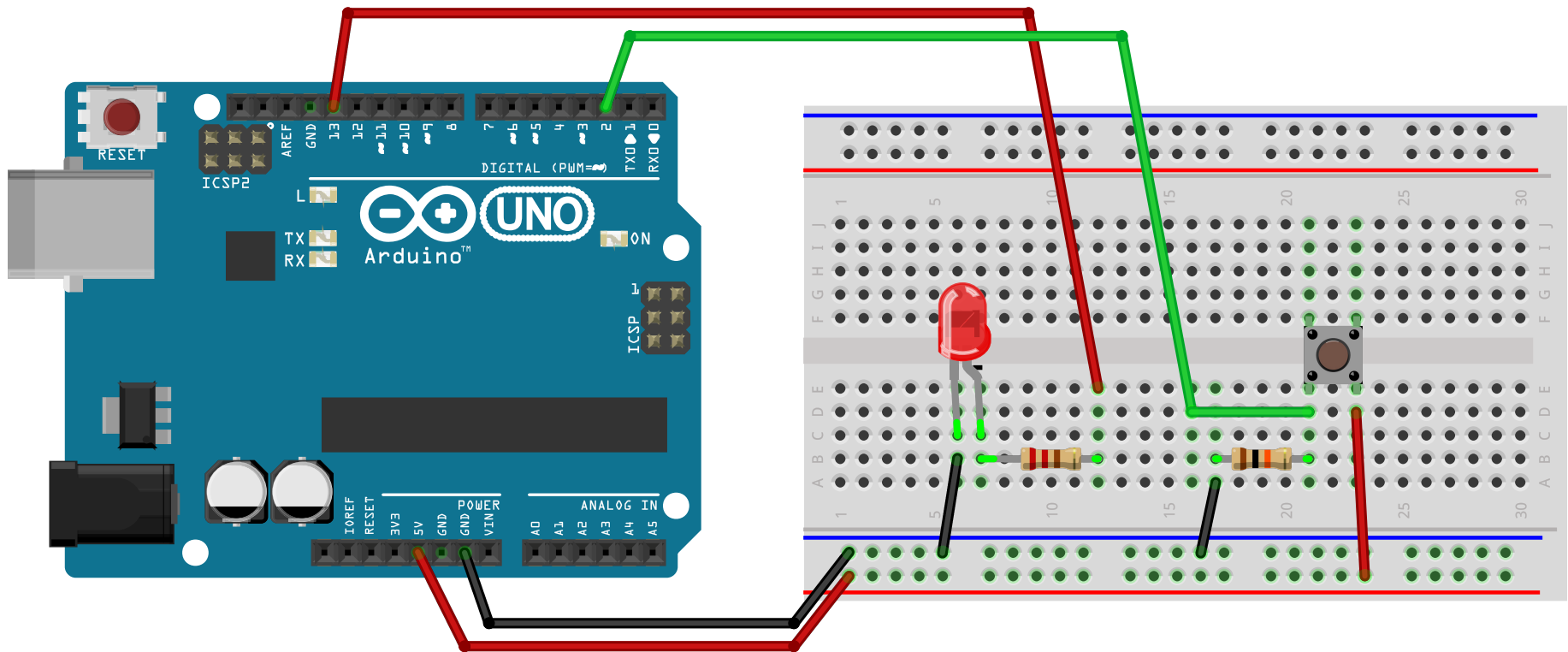
void setup() {
  // initialize the LED pin as an output:
  pinMode(ledPin, OUTPUT);
  // initialize the pushbutton pin as an input:
  pinMode(buttonPin, INPUT);
}

void loop(){
  // read the state of the pushbutton value:
  buttonState = digitalRead(buttonPin);

  // check if the pushbutton is pressed.
  // if it is, the buttonState is HIGH:
  if (buttonState == HIGH) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
  }
}
```

INPUT/OUTPUT EXAMPLE: BUTTON-LED

(the circuit part will be presented and discussed
in detail in lab sessions)



fritzing

PIN (MULTIPLEXED) FUNCTIONS

- Besides being general purpose, some pins have a further function that can be configured/activated:
 - pins 0 and 1 => **TTL** serial interface
 - pins 2 and 3 => **interrupts**
 - pins 3,5,6,9,10 e 11 => **PWM**
 - pins 10, 11, 12, 13 => **SPI bus**
 - pins 13 => **Builtin led**
- Analog pin
 - pin A4 and A5 => **I2C bus**

ANALOG SIGNALS

- A digital signal on a (digital) pin can assume only two logical values: HIGH (corresponding to VCC) o LOW (corresponding to GND), that is 1 or 0
- Instead, an analog signal on a (analog) pin can assume any value in the voltage range GND-VCC (e.g. 0-5 V)
- So, in order to process with a digital computer an analog signal, it must be first converted
- The conversion is made by an electronic component called **ADC (Analog-to-Digital Converter)**, that maps a voltage continuous value in GND..VCC into a discrete value in a certain range (e.g. 0..1023), depending on the number of bits (**resolution**) used by the ADC
 - e.g. by using 10 bits, the continuous value is mapped into a discrete value in the range 0...1023
 - the resolution defines the **precision** of the ADC

READING ANALOG PINS IN WIRING

- Arduino UNO can manage up to 6 input analog signals by means of 6 analog pins, each using a 10-bit ADC
 - pin A0-A5
- The Wiring API to read the value of an analog pin is:

```
int analogRead(int PIN)
```

- internally, this procedure (function) accesses the registers of the C port
- The library provides also a further utility function, **map**, to map a value from a range (e.g. 0...1023) into another application-level range (e.g. 0..255)

EXAMPLE: USING A POTENTIOMETER

```
const int analogInPin = A0; // Analog input pin that the potentiometer is attached to
const int analogOutPin = 9; // Analog output pin that the LED is attached to

int sensorValue = 0;          // value read from the pot
int outputValue = 0;          // value output to the PWM (analog out)

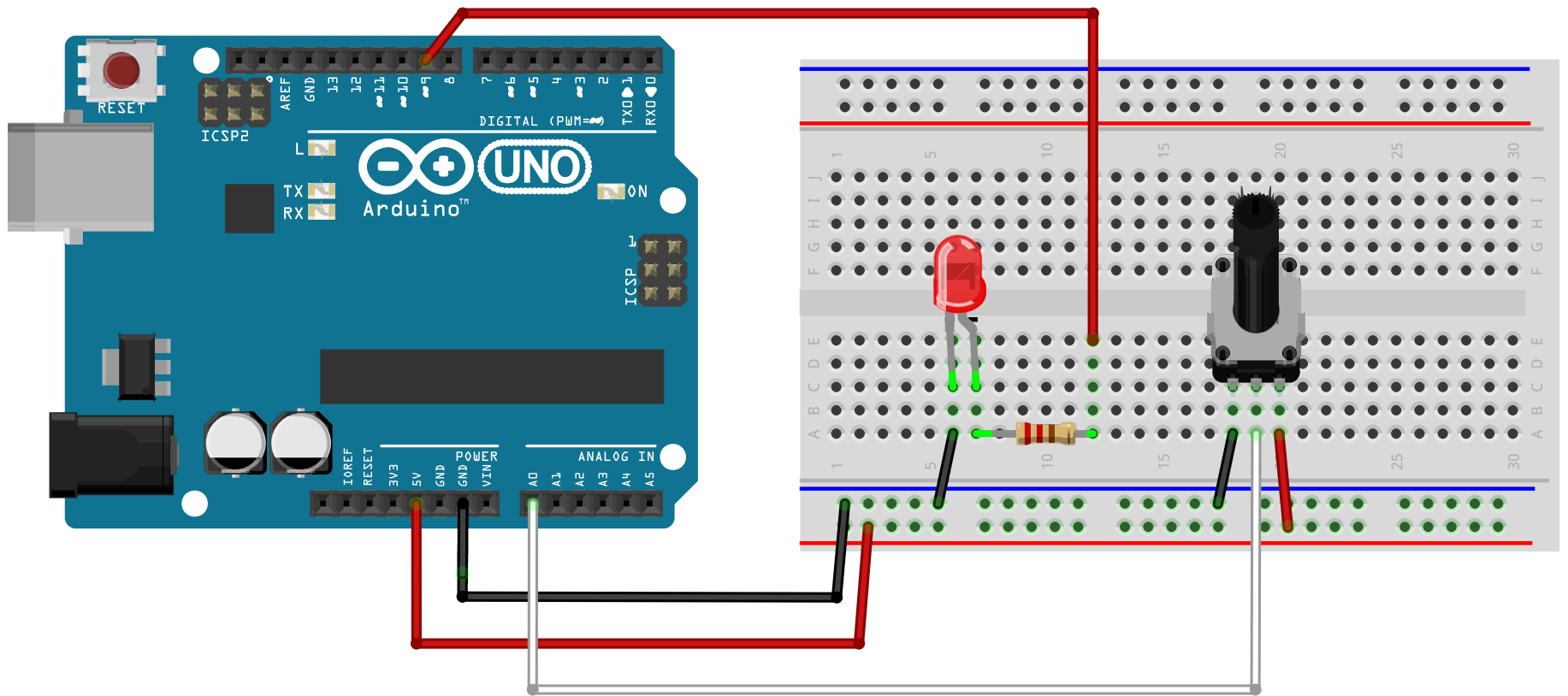
void setup() {
  // initialize serial communications at 9600 bps:
  Serial.begin(9600);
}

void loop() {
  // read the analog in value:
  sensorValue = analogRead(analogInPin);
  // map it to the range of the analog out:
  outputValue = map(sensorValue, 0, 1023, 0, 255);
  // change the analog out value:
  analogWrite(analogOutPin, outputValue);

  // print the results to the serial monitor:
  Serial.print("sensor = "); Serial.print(sensorValue);
  Serial.print("\t output = "); Serial.println(outputValue);

  // wait 2 milliseconds before the next loop
  // for the analog-to-digital converter to settle
  // after the last reading:
  delay(2);
}
```

READING A POTENTIOMETER: THE SKETCH



fritzing

NOTE ON THE CODE: LOGGING THROUGH THE SERIAL

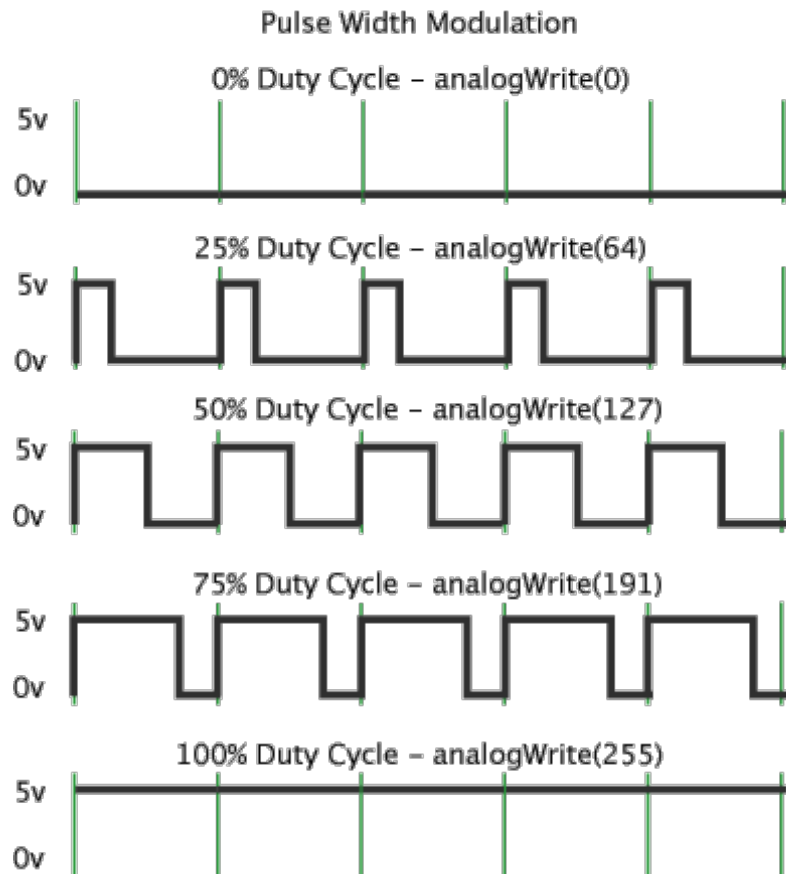
- In the code, the **println** method of the **Serial** class is used to send text messages through the serial port to the host PC, connected to Arduino
- In Arduino IDE, such messages can be visualised by means of the Serial Monitor
 - details about the serial port and its programming will be given later on in this module

PULSEWIDTH MODULATION (PWM)

- PWM is a method to **emulate an analog output signal** on digital pins, through the generation of a periodic square wave
 - i.e. a signal periodically going HIGH and LOW
- The output voltage emulated is defined by the duty cycle of the signal
 - duty cycle: percentage of time in which the signal is HIGH over the time in which the signal is LOW
- The average voltage value is equal to the duty-cycle * VCC (a value in $0 \dots VCC$) of
 - e.g. 50% duty cycle $\Rightarrow 0.5 * VCC$ - e.g. 2.5V for $VCC = 5V$
- Such an emulation approach doesn't work with every output analog device / actuator
 - it works to drive LEDS (controlling the intensity), direct current motors (controlling the speed)

PWM ON ARDUINO

- Pins 3,5,6,9,10 e 11 can be used for PWM output (8 bit)
- API: **analogWrite()**

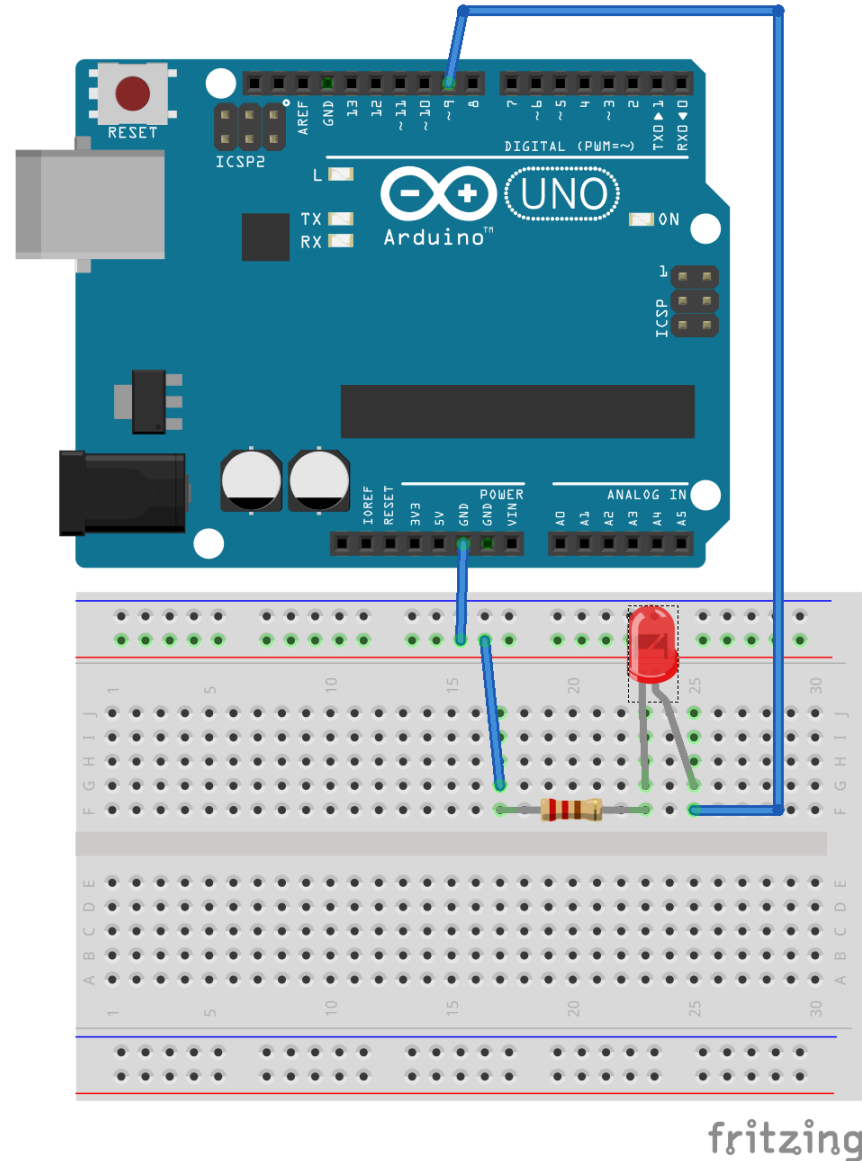


AN EXAMPLE: FADING OF A LED

```
int led = 9;           // the pin that the LED is attached to
int brightness = 0;    // how bright the LED is
int fadeAmount = 5;    // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup() {
  // declare pin 9 to be an output:
  pinMode(led, OUTPUT);
}
// the loop routine runs over and over again forever:
void loop() {
  // set the brightness of pin 9:
  analogWrite(led, brightness);
  // change the brightness for next time through the loop:
  brightness = brightness + fadeAmount;
  // reverse the direction of the fading at the ends of the fade:
  if (brightness == 0 || brightness == 255) {
    fadeAmount = -fadeAmount ;
  }
  // wait for 30 milliseconds to see the dimming effect
  delay(30);
}
```

FADE: SCHEMA



I/O AND INTERRUPTS

- Interrupts are the fundamental mechanism used by the microcontroller *to react to events triggered by external devices*
 - including sensors perceiving the physical world, timers
- Generally speaking, a CPU provides one or multiple pins — called **IRQ** (*Interrupt Request*) — to receive interrupt signals
 - when an interrupt request is received, the CPU suspends the execution of current instructions (e.g. the super-loop), it saves on a stack the address of the next instruction to be executed, and jumps (transfers the control) to the routine (procedure) configured to react to the event - called **interrupt handler** or **interrupt service routine (ISR)**
 - the ISR address is stored into a table called interrupt table or interrupt vectors
- In microcontrollers, IRQ are typically connected to one or multiple GPIO

I/O AND INTERRUPTS: ATmega328P

- The ATmega328P has 2 pins - **pin 2 and pin 3** - that can be configured to generate (to the CPU) *external interrupts* (*)
- In particular, the pin can be configured to generate an interrupt on different conditions
 - when there is a RISING or FALLING edge
 - when there is a CHANGE
 - when the signal is LOW

[*] actually it is possible to enable and manage interrupts for all pins, but limiting to a specific kind of interrupt: PIN CHANGE (vs. EXTERNAL)

<https://github.com/GreyGnome/EnableInterrupt>

ATMega328P: INTERRUPT VECTORS

11.4 Interrupt Vectors in ATmega328 and ATmega328P

Table 11-6. Reset and Interrupt Vectors in ATmega328 and ATmega328P

VectorNo.	Program Address ⁽²⁾	Source	Interrupt Definition
1	0x0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event

HANDLING INTERRUPTS IN WIRING

- In Wiring it is possible to directly specify and attach an ISR by means of the routine:

```
attachInterrupt(intNum, ISR, mode) ;
```

- Parameters:
 - intNum: interrupt number (int)
 - ISR: pointer to the interrupt handler
(tipo: void (*func)(void))
 - mode: when the interrupt should be generated:
CHANGE/FALLING/RISING: at every change HIGH -> LOW,
LOW -> HIGH
LOW/HIGH: when the state is LOW or HIGH
- The utility function `digitalPinToInterrupt(numPin)` makes it possible to get the number of the interrupt (see interrupt vectors) given the specified pin

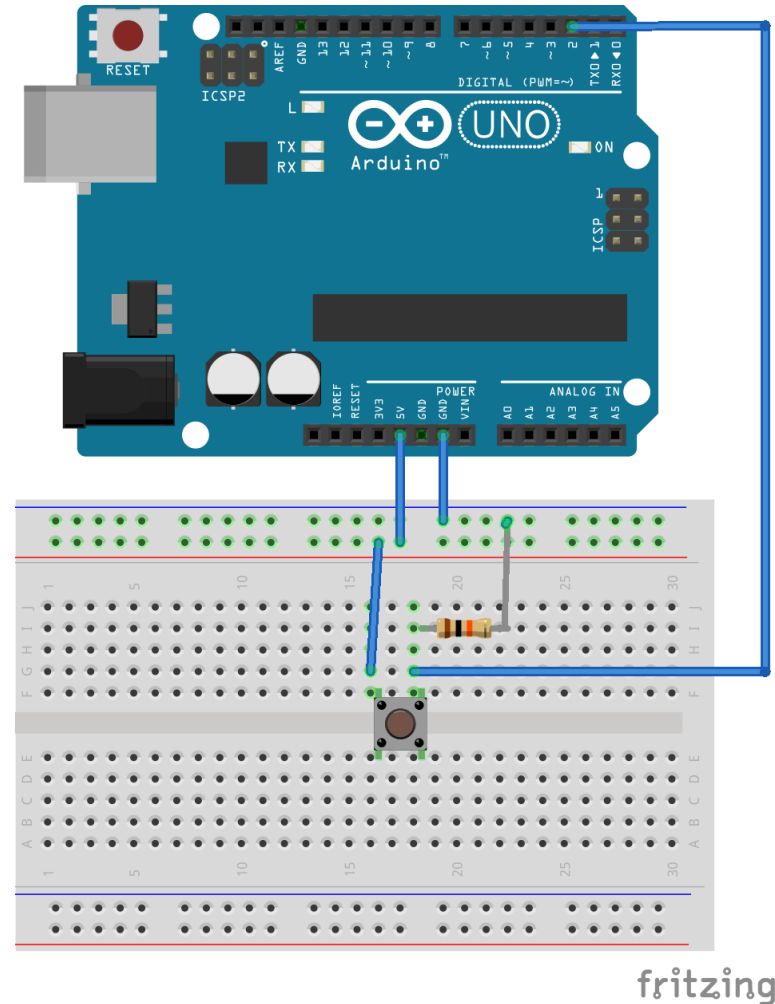
EXAMPLE

```
volatile int count = 0;

void setup()
{
    // 0 => detecting pin 2 changes
    attachInterrupt(0, inc, CHANGE);
    Serial.begin(9600);
}

void loop()
{
    Serial.println(count);
}

void inc()
{
    count++;
}
```



fritzing

BEWARE - concurrency problems here

- race conditions

DISABLING INTERRUPTS

- Possibility to disable/enable interrupt from the user programs, typically to avoid concurrency problems
 - by setting a bit in the status register of the CPU
- Instructions
 - STI = Set Interrupt (enable)
 - CLI = Clear Interrupt (disable)
- In Wiring/Arduino:
 - **noInterrupts()**
 - **interrupts()**

THE EXAMPLE REVISED

```
#define BUTTON_PIN 2
volatile int count = 0;
int prev = 0;

void setup()
{
    Serial.begin(9600);
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), inc, RISING);
}

void loop() {
    noInterrupts();
    int current = count;
    interrupts();
    if (current != prev){
        Serial.println(current);
        prev = current;
    }
}

void inc() { count++; }
```

INTERRUPT HANDLER DESIGN

- When interrupts are disabled, *the system is no more reactive to external events*
- For this reason, the design of an interrupt handler should be ruled by a careful discipline
 - **interrupt handlers should be short time**
 - they cannot block or execute infinite loops
 - we want **low interrupt latency**
 - interrupt latency = amount of time used to react to an interrupt
 - => minimising the time period in which interrupts are disabled
 - => using buffers and producers/consumers strategy to do long-term / heavy computational tasks in reaction to interrupts (when needed)
 - the interrupt handler (producer) produces a task in a buffer which is fetched and executed later on by the main loop / super loop

INTERRUPTS FAQ

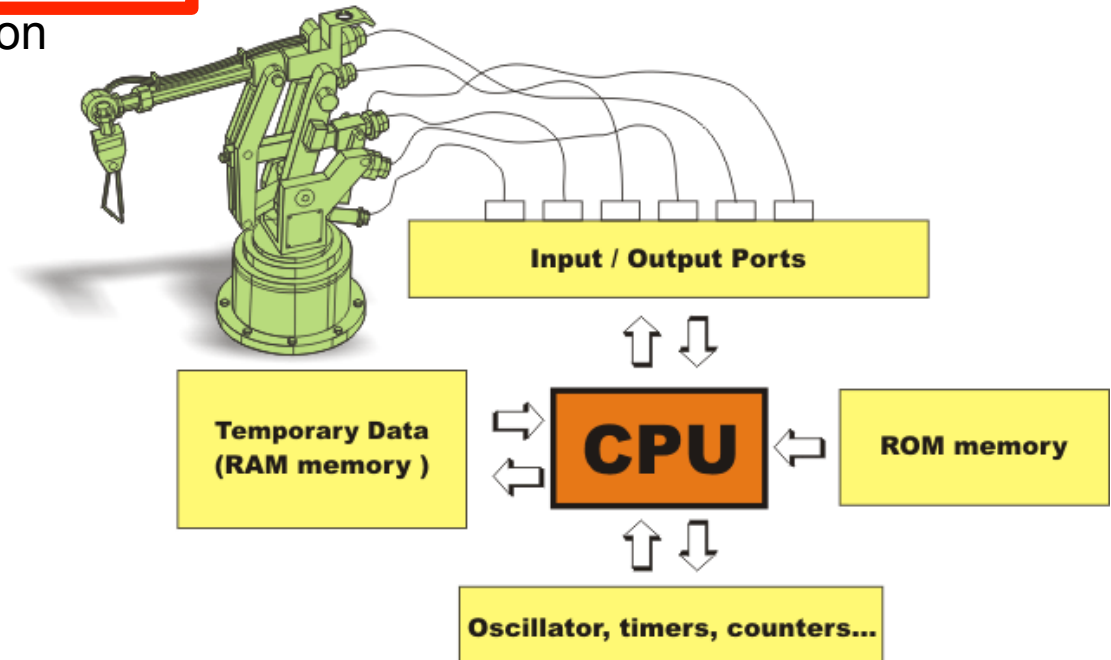
- *Can a processor being interrupted when executing a single instruction?*
 - no, typically machine language instructions are executed atomically (modulo complex instructions that account for multiple memory transfers)
- *If two interrupts occur in the same time, which one is served?*
 - every CPU assigns some priority to interrupts, so that it executes always the interrupt with higher priority
- *About interrupt nesting: can an interrupt request interrupt the execution of an interrupt handler?*
 - in most processors YES, so we need to disable interrupt in the interrupt handler (and re-enable them)
 - in AtMega328P / AVR8 NO, it does not support interrupt nesting
- *What happen to interrupt requests when interrupt are disabled?*
 - typically a CPU keeps track of such requests and execute them as soon as the interrupts are enabled again
 - AVR8 does not track them
- *What happens if we disable interrupts and we forgot to enable them again?*
 - system malfunctioning, crash
- *What happens if we disable interrupts multiple times (before re-enabling)?*
 - nothing, they are idem-potent

INTERRUPTS ON ARDUINO: NOTES

- On AVR / Arduino, when the interrupt handler is executed, interrupts are disabled
 - it does not support interrupt nesting
- **CONSEQUENCE/WARNING:** *library functions/procedures whose function depends on interrupts cannot be used in interrupt handlers*
 - An example: **delay()**
 - it is based on **millis()** which uses (timer) interrupts
 - conversely **delayMicroseconds()** works, since it is not based on timers
 - it is based on the execution of instructions, knowing their duration in clock cycles
 - <https://www.arduino.cc/en/Reference/DelayMicroseconds>

MICROCONTROLLER ELEMENTS

- CPU
- Memory unit
- Ports for general purpose Input/Output (GPIO)
- Analog/digital converters
- **Timers**
- Bus for serial communication
- Clock
- Power circuit



TIMERS

- *Timers* play a fundamental role to realise **time-oriented behaviour**, in which time becomes an important aspect in task specification. A timer can be used:
 - measure time intervals
 - to create PWM signals
 - to create timeouts and alarms
 - to create multi-tasking based on preemptive scheduling
- A timer can be represented as a *simple counter* which is incremented at the HW level at some frequency.
- Programmable timers allow for
 - configure the frequency
 - access and read the current value of the counter
 - attach interrupts to implement time-triggered behaviours based on time events

ATMega328 EXAMPLE

- The ATMega328 has 3 internal timers
 - timer0
 - timer1
 - timer2
- Each timer has a counter which is incremented each clock cycle (of the timer)
 - timer0 and timer2 have 8-bit counters
 - timer1 has a 16-bit counter

API USING TIMERS

- Timers are exploited to implement high-level functionalities provided by Wiring. Examples:
 - **millis()** procedure, using time 0

```
unsigned long millis()
{
    unsigned long m;
    uint8_t oldSREG = SREG;

    cli();
    m = timer0_millis;
    SREG = oldSREG;

    return m;
}
```

NOTE

- timer0_millis = variable that contains the value of the timer0 counter
- interrupts disabled to avoid race conditions

- **analogWrite()** procedure, for PWM signals

TIMER AND INTERRUPTS ON ATmega328

- Different working modalities for managing interrupts in timers
 - one is **CTC (Clear Timer on Compare Match)**
- In CTC modality, timer interrupts are generated when the counter reaches a certain value
 - this value must be stored in a register called **compare match register**
 - When the timer counter reaches this value, the counter is reset and starts again counting
- By choosing the value of the CTC register, we specify interrupts frequency

TIMER AND INTERRUPTS ON ATmega328: DETAILS

- Counters are updated at 16 MHz
 - so each tick occurs every $1/16,000,000$ of a second ($\sim 63\text{ns}$),
- timer0 and timer2 are 8 bit => they take $256 \cdot 63 = 16128 \text{ ns} = 16.1 \text{ us}$ to go in overflow (e restart to 0)
- timer1 is 16 bit => it takes $65536 \cdot 63 = 4128768 \text{ ns} \sim 4129 \text{ us} \sim 4.1 \text{ ms}$
- This frequency can be modulated by specifying a *prescaler value*, which essentially functions as divider of the original frequency(16 MHz):

`timer speed (Hz) = (Arduino clock speed (16MHz)) / prescaler`

- So e.g. by specifying a prescaler = 1 => counter incremented at 16MHz, prescaler = 8 => incremented at 2 MHz, .. prescaler = 1024 => 16 KHz.
 - the value of the prescaler can be 1, 8, 64, 256, and 1024.

TIMER AND INTERRUPTS ON ATmega328: DETAILS

- Given the prescaler, then the interrupt generation frequency is:

$$\text{desired interrupt frequency (Hz)} = \frac{16,000,000\text{Hz}}{(\text{prescaler} * (\text{compare match register} + 1))}$$

(+1 since the value 0 in the CTC is the first significant value)

- Then, given the desired interrupt frequency, the CTC register can be set using the formula:

$$\text{compare match register} = \left(\frac{16,000,000\text{Hz}}{(\text{prescaler} * \text{desired interrupt freq})} \right) - 1$$

- Checking overflows
 - for timers 0 and 2, this value must be less than 256
 - for timer 1 the value must be less than 65536
 - if there is overflow, then we have to increase the prescaler

TIMER AND INTERRUPTS ON ATmega328: DETAILS

- Example: suppose that we want 1 interrupt per second (1 Hz). Then:

`compare match register = (16,000,000 / (prescaler * 1)) - 1`

- if we use a prescaler = 1024, then we get:

`compare match register = (16,000,000 / (1024 * 1)) - 1 = 15,624`

- Since the value is greater than 256 but less than 65536, then we can use timer 1.

REGISTERS CONFIGURATION

- Each timer has 2 registers to store timer configuration: **TCCRxA**, **TCCRxB** (where x = timer number 1..3)
 - 8-bit registers
 - the most important bits are the 3 least significant ones CSx2, CSx1, CSx0, determining timer clock configuration

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$\text{clk}_{\text{I/O}} / (\text{No prescaling})$
0	1	0	$\text{clk}_{\text{I/O}} / 8$ (From prescaler)
0	1	1	$\text{clk}_{\text{I/O}} / 64$ (From prescaler)
1	0	0	$\text{clk}_{\text{I/O}} / 256$ (From prescaler)
1	0	1	$\text{clk}_{\text{I/O}} / 1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

- CTC modality is enabled by activating specific bit in TCCRxB
- CTC register is labelled as OCRxA

ARDUINO RAW EXAMPLE

```
void setup(){
  Serial.begin(9600);
  setupTimer(1000);
}

volatile boolean timerFlag = false;

ISR(TIMER1_COMPA_vect){
  timerFlag = true;
}

int count = 0;

void step()
{
  count++;
  if (count % 100 == 0){
    Serial.println(count);
  }
}

void loop(){
  /* wait for timer signal */
  while (!timerFlag){}
  timerFlag = false;
  step();
}
```

```
/* setup the timer1 (16 bit) to
 * a specific freq
 */
void setupTimer(int freq){
  // disabling interrupt
  cli();
  TCCR1A = 0; // set entire TCCR1A register to 0
  TCCR1B = 0; // same for TCCR1B
  TCNT1 = 0; //initialize counter value to 0
  /*
   * set compare match register
   * OCR1A = (16*2^20) / (100*PRESCALER) - 1
   * (must be < 65536)
   * by assuming a prescaler = 1024, then
   * OCR1A = (16*2^10)/freq
   */
  OCR1A = 16*1024/freq;
  // turn on CTC mode
  TCCR1B |= (1 << WGM12);
  // Set CS11 for 1024 prescaler
  TCCR1B |= (1 << CS12) | (1 << CS10);
  // enable timer compare interrupt
  TIMSK1 |= (1 << OCIE1A);
  // enabling interrupt
  sei();
}
```

REMARKS

- In the example, timer1 (16 bit) is configured to generate a tick (then an interrupt) 100 times per second (100 Hz).
- In the main loop, at each tick a count is incremented and sent to serial line every 100 increments
 - so a message per second is printed
- **ISR** is a macro used to directly declare the interrupt handler
- **volatile** attribute in timerFlag

TIMER LIBS

- Different timer libraries available for Arduino, that allow for easing the management of the timer
- A main one is **TimerOne**
 - <https://code.google.com/p/arduino-timerone>
 - it provides classes that allow to configure the times and to attach interrupt routine

AN EXAMPLE: TIME-DRIVEN BLINKING

```
#include<TimerOne.h>

const int LED_PIN = 13;
boolean flagState = false;

void blinkHandler(){
    if (!flagState){
        digitalWrite(LED_PIN, HIGH);
    } else {
        digitalWrite(LED_PIN, LOW);
    }
    flagState = !flagState;
}

void setup()
{
    pinMode(LED_PIN,OUTPUT);
    Timer1.initialize(1000000); // set a timer of length 10^6 micro sec = 1 sec
    Timer1.attachInterrupt(blinkHandler); // runs blinkHandler on each timer interrupt
}

void loop(){}
}
```

THE WATCH DOG TIMER

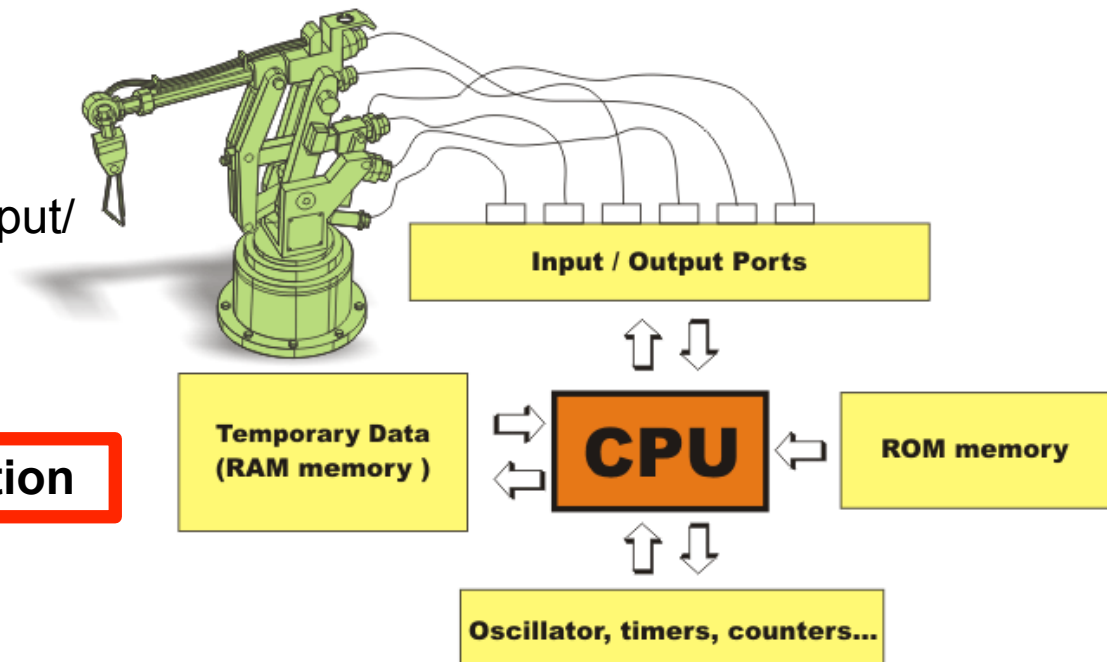
- It is a timer that counts until some specified value, after than a reset signal is produced to reboot the system
 - in normal functioning, the watch dog periodically receives a signal before reaching the threshold, resetting the counter
 - if no signal is received on time, than it means that the microprocessor encountered a problem (e.g. deadlock) and it is reset
- Widespread in embedded systems

TIMERS AND PWM

- In the lib implementation managing the GPIO in Wiring for Arduino, timers are used to realise PWM output signals
 - Timer0 is used for pin 5 and 6
 - Timer1 is used for pins 9 and 10
 - Timer2 is used for pins 11 e 3
- **We need to pay attention to interferences when using timers and PWM capability**
 - e.g. if we use Timer1 directly, then we cannot use pin 9 and 10 for PMW
 - this could happen suddenly when using libraries that may use PWM techniques
 - e.g. libraries for managing servo motors

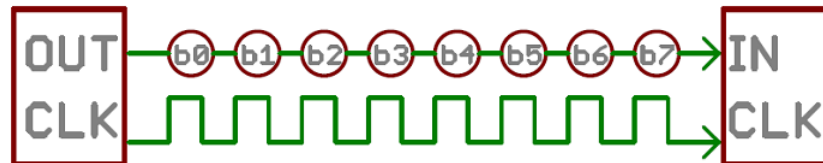
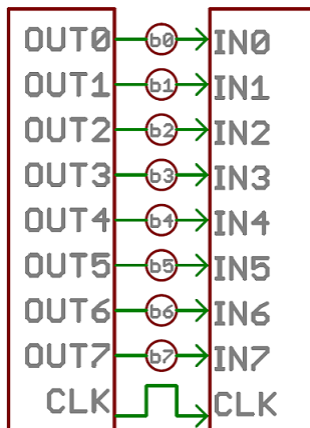
MICROCONTROLLER ELEMENTS

- CPU
- Memory unit
- Ports for general purpose Input/Output (GPIO)
- Analog/digital converters
- Timers
- **Bus for serial communication**
- Clock
- Power circuit



BUSES AND PROTOCOLS

- Protocols to exchange data through single pins can become complex depending on the external communicating I/O device
- To this purpose, specific communication interfaces, buses and protocols have been introduced
- Two main kinds: **serial** and **parallel**
 - *parallel* interfaces allow for transferring multiple bits at the same time, by means of multiple lines
 - 8-bit, 16-bit, 32-bit bus
 - *serial* interfaces send and receive streams of bits in a sequential way, using a single line/channel



SERIAL INTERFACES/BUSES

- Two main categories
 - **asynchronous**
 - no clock line is used for synchronisation
 - two lines: TX to transmit, RX to receive
 - examples: **USB**, **RS-232** e **TTL**
 - oldest and simpler protocol
 - **synchronous**
 - a clock signal is used to synchronise the communication among the parts
 - this allows to achieve higher speed than asynchronous serial
 - examples: **I²C** and **SPI** synchronous serial buses

ASYNCHRONOUS SERIAL

- In serial asynchronous interface, since there is not a specific clock line, the synchronisation is achieved through a ***protocol***, to ensure an error-free and robust bit transmission
- The protocol can be configured by specifying some parameters, that must be adopted by both the communicating parts:
 - **baud rate**
 - **data frame**
 - **synchronous bits**
 - **parity bit**

BAUD RATE

- Transmission speed in **bits per seconds**
- A typical value used is 9600 bps
 - other standard values are 1200, 2400, 4800, 19200, 38400, 57600, and 115200
- The higher is the baud rate, the higher is the speed
 - however there is a limit (typically 115200), related to microcontrollers capabilities
 - using higher values could lead to transmission errors

DATA FRAME

- Each data block - typically one byte - is transmitted inside a (bit) packet called frame. These frames are created adding to the data block also a synchronization bit and a parity bit:



- The data chunk (block) represents the real data transmitted: each packet can have a data chunk from 5 to 9 bit (typical value: 8)
- About little endian or big endian
 - part of the protocol includes agreeing also on data *endianness* dei dati, i.e. the order used to send the bit (from the most significative or the less significative).
 - default if not specified: less significative
- Synchronisation bits include a start bit and a stop bit, to mark the beginning and end of the packet (there is always a single start bit, while stop bits can be 2).
- Parity bits (optional) are used for simple low-level error checking.

CONCRETE EXAMPLE

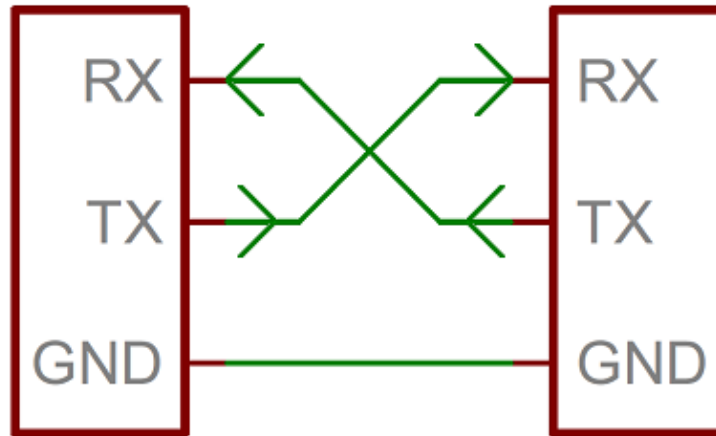
- II **9600 8N1** - 9600 baud, 8 data bits, no parity, and 1 stop bit - is one of the most used protocol
- Transmission example
 - suppose to transmit ASCII characters 'O' and 'K'. ASCII value of O is 79, i.e. 01001111. The value of K in binary is 01001011. Bits are sent starting from the least significative:



- since we are transferring at 9600 bps, the time used to keep the signal high (or low) is $1/(9600 \text{ bps})$, i.e. $104 \mu\text{s}$ per bit.
- for each byte there are 10 bits sent: one start bit, 8 data bits, and one stop bit => at 9600 bps we are sending 960 ($9600/10$) byte per sec.

SERIAL HARDWARE

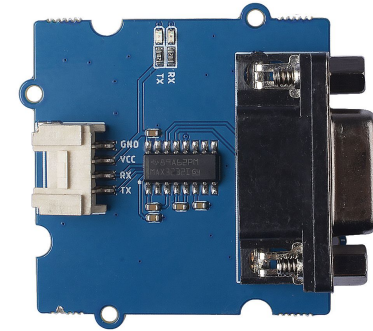
- At the hardware level, an async serial bus is composed by two lines (wires):
 - one to transmit bits
 - one to receive bits
- Two pins are needed then - one to receive (RX) and one to transmit (TX)



- RX and TX names are referred to the device itself
 - RX pin of one device must be connected to the TX pin of the other device

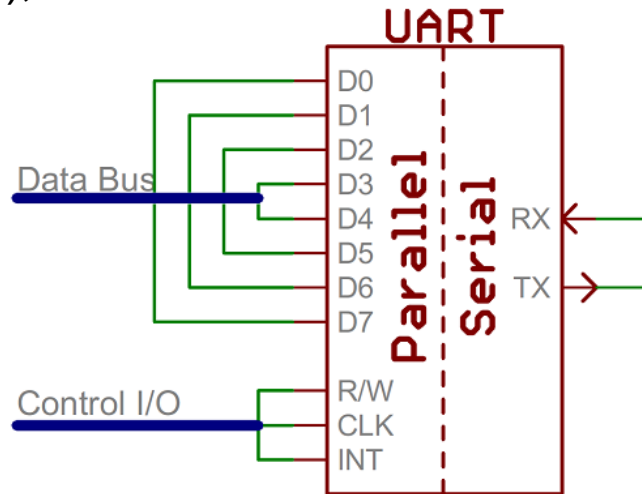
HARDWARE IMPLEMENTATIONS OF SERIAL

- **RS-232**
 - first HW serial communication protocol widely used as a communication method for computers and devices by serial ports
- **TTL**
 - Transistor-Transistor Logic
 - a serial communication commonly found in UART (universally asynchronous receiver/transmitter) transmission method, very common in microcontrollers
 - used in Arduino



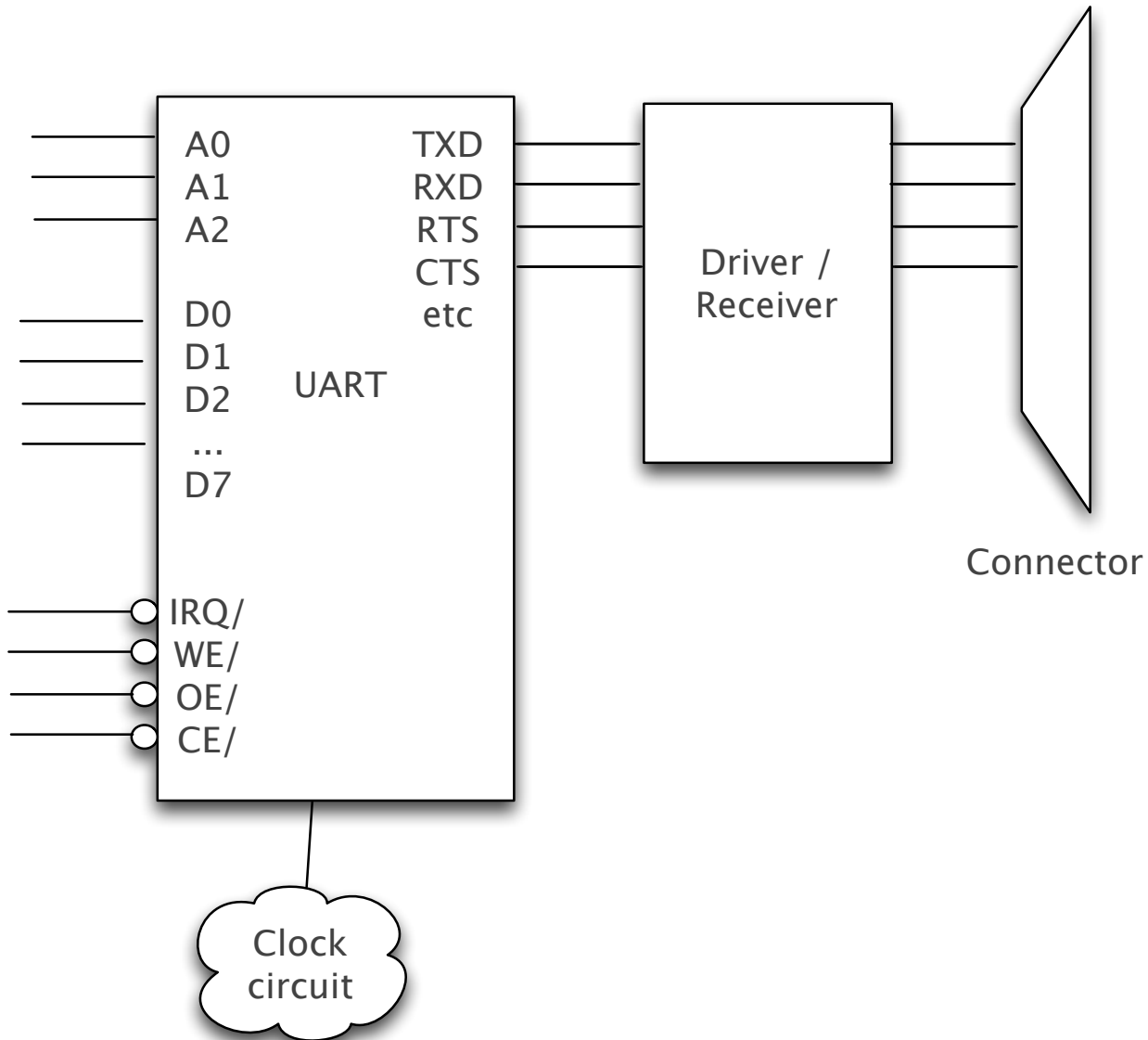
UART

- A main component of a serial system is **UART** (universal asynchronous receiver/transmitter), that is the circuit block that manage the data to and from the serial interface.
- It functions as a converter between the parallel interface (data bus) and the serial interface
 - on one side of the UART there is a 8-bit (ore more) data bus (plus some control pin), on the other side there are the 2 serial lines



- UARTs exist also as stand-alone integrate circuits, however typically they are embedded inside microcontrollers
 - example: Arduino Uno - based on ATmega328 - has a single UART, while Arduino Mega - based on su ATmega2560 - has 4

UART



OTHER ASPECTS

- Full-duplex o half-duplex
 - a serial interface in which both the devices can send and receive data can be either **full-duplex** or **half-duplex**
 - full-duplex if both the devices can simultaneously transmit and receive (different data lines)
 - half-duplex if the devices can send or receive in turn
- Single-wire
 - some serial bus can have a single wire, between a device which always send and a device which always receive
 - for instance: Serial Enable LCD

ASYNCR SERIAL ON ARDUINO

- Arduino UNO has one serial port **hardware TTL**, multiplexed on GPIO 0 e 1
 - **pin 0 (RX)** is used to receive, **pin 1 (TX)** to transmit
 - these pins are connected to the corresponding ATmega8U2 USB-to-TTL-serial chip pins
 - this converts a serial TTL into an USB and viceversa
 - these pins are used for more than one function
 - serial and GPIO
 - they are mutually exclusive — when using for one function, the other does not work
- **Serial library/class in Wiring**
 - full access and control of the (async) serial
 - can be used to send and receive messages and raw data to the system connected via USB

SERIAL LIBRARY

- Serial class is an extension of Stream and includes methods such as:
 - begin(), end()
 - to begin and end the communication session
 - read(), readBytes(), readBytesUntil(), peek(), available()
 - to receive data
 - parseFloat(), parseInt()
 - write(), flush()
 - to transmit data
 - print(), println()
 - to transmit strings
 - setTimeout()
 - serialEvent()

EXAMPLE

- Echo program

```
char data;

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  if (Serial.available()){
    data = Serial.read();
    Serial.print(data);
  }
}
```

Using e.g. ArduinoIDE SerialMonitor, it is possible to send characters that are read and re-sent to the connected PC

SYNCHRONOUS SERIAL

- Synchronous serial interfaces use a clock signal to synchronise and one or multiple lines for sending/receiving data
 - more complex than serial async, but typically higher speed
 - main examples: SPI e I²C

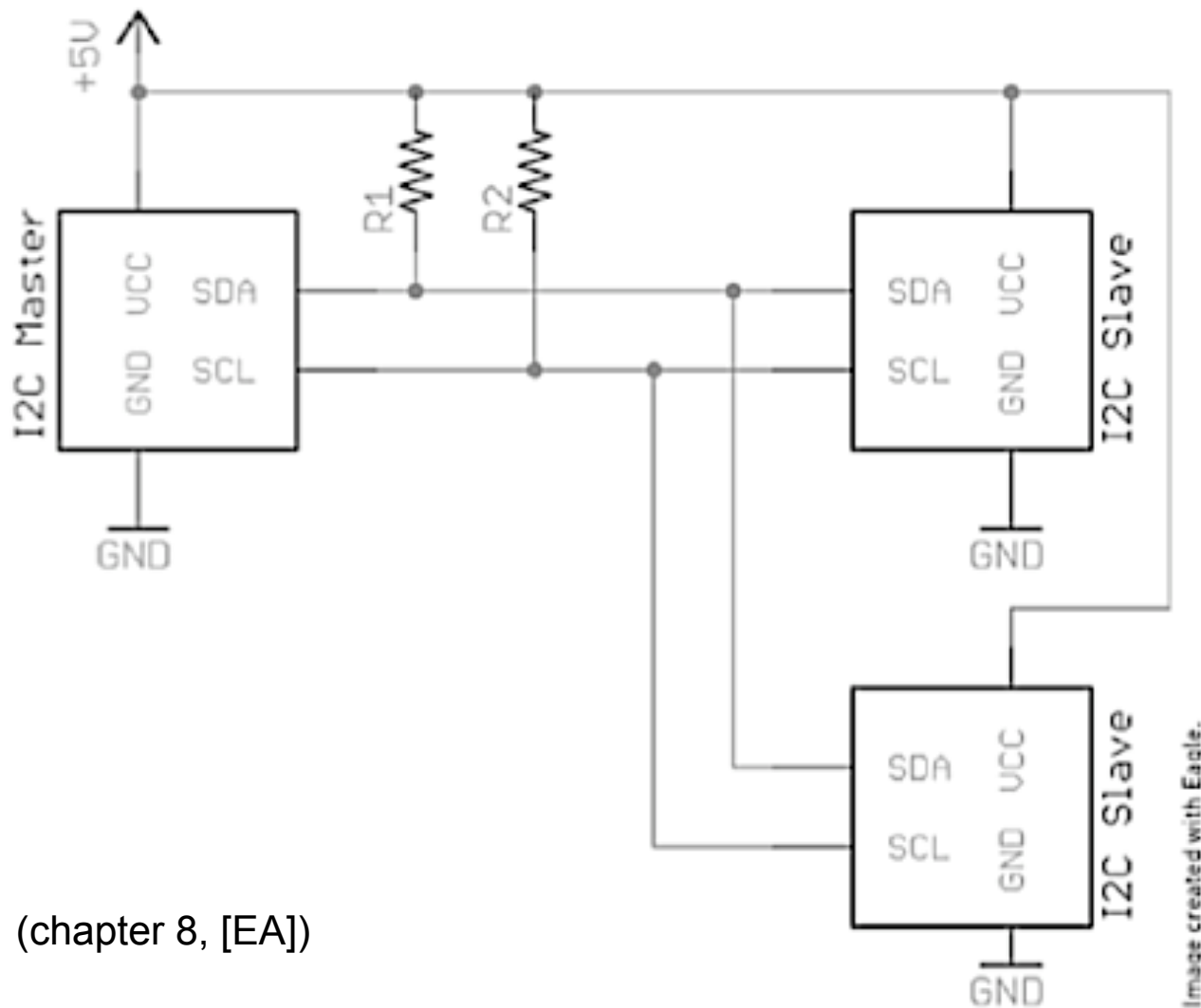
I²C BUS/PROTOCOL

- I²C ("eye squared see" o "eye two see") is a standard bus/protocol
 - good speed, minimising the number of pins required: only 2
 - introduced by Philips in '80s, then standardised inn '90
 - also called "Two-wire" (TW) since it uses 2 lines for the communication (data, clock)
- Specs
 - 7-bit or 10-bit bus
 - bus speed achieves 100 kbit/s in *standard mode* e 10 kbit/s in *low-speed mode*
 - recent version have a *fast mode* at 400 kbit/s, andf fast mode plus (1Mbit/s) and High-Speed mode (3.4Mbit/s)

I²C ARCHITECTURE

- **Master-slave architecture**
 - the I²C bus is controlled by a master device
 - typically the microcontroller
 - it includes one or multiple slave devices that receive info from the master
- All devices share two lines
 - a signal clock (*Serial Clock Line*, **SCL**), necessary to synchronise the communication
 - a bidirectional data line (*Serial Data Line*, **SDA**) to send and receive data

I²C ARCHITECTURE



(chapter 8, [EA])

Figure 8-1: I²C reference hardware configuration

MASTER-SLAVE PROTOCOL

- Communication is always started by the master
 - slaves can only respond
- Commands sent by the master on the bus are received by all slaves
 - each slave has its own 7 bit unique ID, which is sent by the master when it starts the communication
 - only the target slave reacts to the message sent by the master
- Typical communication schema:
 - Master sends start bit
 - Master sends the 7-bit id of the slave device
 - Master sends read (0) or write (1) bit, either if it wants to write data on device registers or read data from
 - Slave responds with an ack (ACK bit, active when it is 0)
 - In write mode, the master sends data as a sequence of bytes and the slave replies with ACKs. In read mode, the master receives a stream of bytes, sending an ACK after each byte received
- When communication is finished, the master sends a STOP bit

I²C IN ARDUINO/ATmega328P - WIRE LIBRARY

- Arduino/ATmega328P natively supports I2C
 - 2 pins: A4 e A5
 - multiplexed pins (used also for analog reads)
- **Wire** library
 - docs: <http://arduino.cc/en/Reference/Wire>
 - class Wire, extension of Stream
 - methods
 - `begin()`
 - `beginTransaction(), endTransmission()`
 - `write(), requestFrom(), read()`
 - `available()`
 - `onReceive(), onRequest()`

I²C EXAMPLE

- Example:
 - using the temperature sensor TC74, which works with I2C
- Notes
 - pin multiplexing: **SDA (data) e SCL (clock) pin are connected to A4 e A5**
 - when the Wire library is initialised, then these pins are used for I2C and cannot be used to analog reads

Package Types

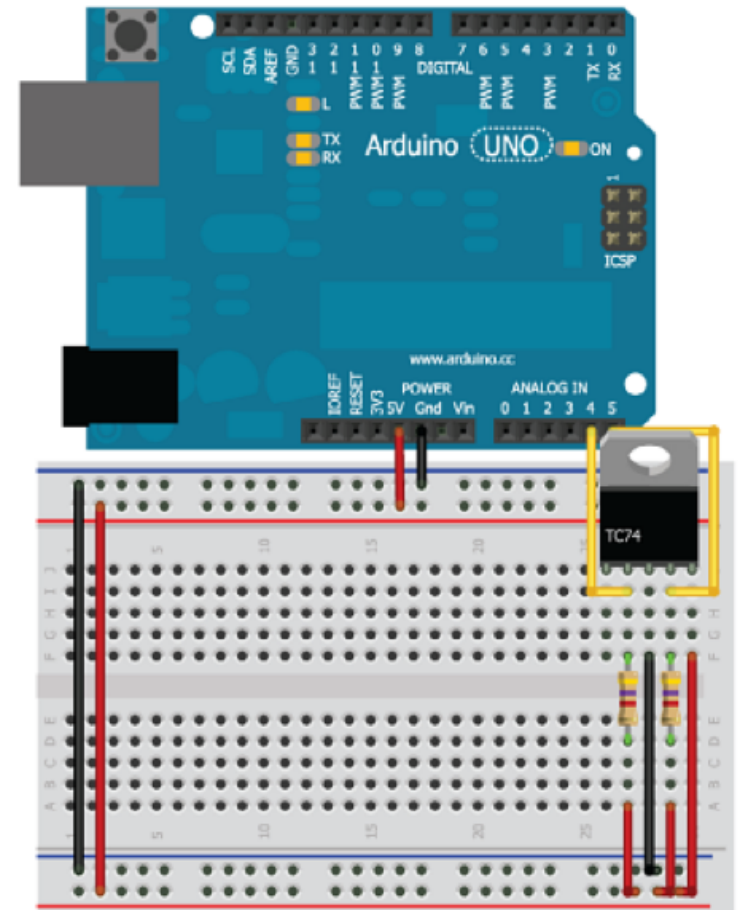
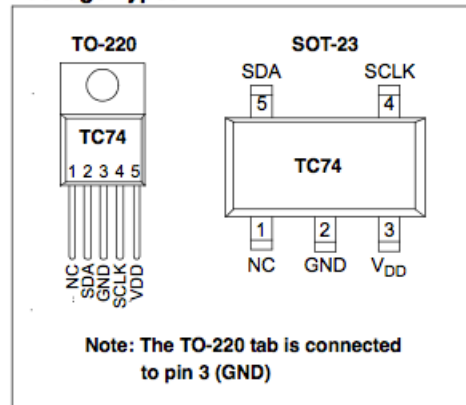


Figure 8-4: Temperature sensor

NOTE: 2 pull-up resistor are needed (e.g. 4.7 KOhm)

Image created with Fritzing.

THE DATASHEET

- Component datasheet
 - <http://ww1.microchip.com/downloads/en/DeviceDoc/21462D.pdf>
- It includes I2C protocol info:

Write Byte Format

S	Address	WR	ACK	Command	ACK	Data	ACK	P
	7 Bits			8 Bits		8 Bits		

Slave Address

Command Byte: selects which register you are writing to.

Data Byte: data goes into the register set by the command byte.

Read Byte Format

S	Address	WR	ACK	Command	ACK	S	Address	RD	ACK	Data	NACK	P
	7 Bits			8 Bits			7 Bits			8 Bits		

Slave Address

Command Byte: selects which register you are reading from.

Slave Address: repeated due to change in data-flow direction.

Data Byte: reads from the register set by the command byte.

Receive Byte Format

S	Address	RD	ACK	Data	NACK	P
	7 Bits			8 Bits		

S = START Condition

P = STOP Condition

Shaded = Slave Transmission

Data Byte: reads data from the register commanded by the last Read Byte or Write Byte transmission.

THE DATASHEET

4.0 REGISTER SET AND PROGRAMMER'S MODEL

TABLE 4-1: COMMAND BYTE DESCRIPTION (SMBUS/I²C READ_BYTE AND WRITE_BYTE)

Command	Code	Function
RTR	00h	Read Temperature (TEMP)
RWCR	01h	Read/Write Configuration (CONFIG)

TABLE 4-3: TEMPERATURE REGISTER (TEMP)

D[7]	D[6]	D[5]	D[4]	D[3]	D[2]	D[1]	D[0]
MSB	X	X	X	X	X	X	LSB

In temperature data registers, each unit value represents one degree (Celsius). The value is in 2's complement binary format such that a reading of 0000 0000b corresponds to 0°C. Examples of this temperature to binary value relationship are shown in Table 4-4.

TABLE 4-4: TEMPERATURE-TO-DIGITAL VALUE CONVERSION (TEMP)

Actual Temperature	Registered Temperature	Binary Hex
+130.00°C	+127°C	0111 1111
+127.00°C	+127°C	0111 1111
+126.50°C	+126°C	0111 1110
+25.25°C	+25°C	0001 1001
+0.50°C	0°C	0000 0000
+0.25°C	0°C	0000 0000
0.00°C	0°C	0000 0000
-0.25°C	-1°C	1111 1111
-0.50°C	-1°C	1111 1111
-0.75°C	-1°C	1111 1111
-1.00°C	-1°C	1111 1111
-25.00°C	-25°C	1110 0111
-25.25°C	-26°C	1110 0110
-54.75°C	-55°C	1100 1001
-55.00°C	-55°C	1100 1001
-65.00°C	-65°C	1011 1111

THE PROTOCOL

- Table 4-1 => 2 registers to be read
 - one stores the temperature in Celsius
 - one the chip config
- Tables 4-3 and 4-4 show hoe the info are stored in the 8-bit register
- Protocol to read the temperature
 - 1.it sends the device id/address in write mode and send a 0 (read cmd)
 - 2.it sends the device id/address in read mode and request a 8 bits (1 byte) data
 - 3.it waits to receive all 8 bits

ARDUINO PROGRAM

```
//Include Wire I2C library
#include <Wire.h>

int temp_address = 72; //1001000 in decimale
void setup()
{
  Serial.begin(9600);
  Wire.begin(); /* lib init */
}

void loop()
{
  Wire.beginTransmission(temp_address);
  Wire.write(0);
  Wire.endTransmission();

  Wire.requestFrom(temp_address, 1);
  while(Wire.available() == 0);
  int c = Wire.read();
  Serial.print(c);
  Serial.print("C ");
  delay(500);
}
```

- Wire.beginTransmission
=> starts the communication session with the slave
- Wire.write
=> sends a 0, specifying that we want to read the value of the data register
- Wire.endTransmission
=> sends a stop to notify that we finished writing

then

- Wire.requestFrom
=> the master waits to receive a byte
- Wire.available()
=> checks if the data has arrived
- Wire.read()
=> reads the value

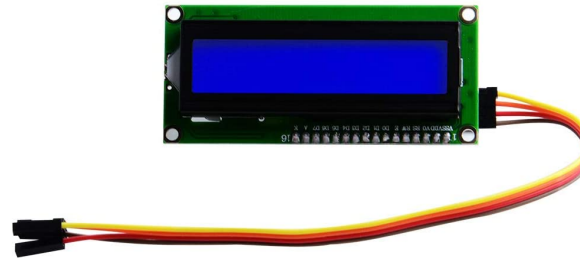
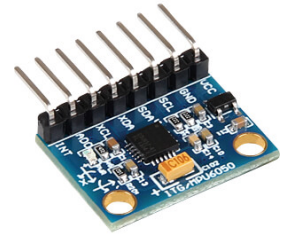
Example taken from “Exploring Arduino” [EA], p. 171

Complete source code on github

<https://github.com/sciguy14/Exploring-Arduino>

OTHER EXAMPLES OF I2C DEVICES

- Sensors
 - MPU6050
 - sensor with compass + accelerometer
 - <http://invensense.com/mems/gyro/mpu6050.html>
 - LSM330
 - sensor with compass + accelerometer
 - http://www.st.com/web/en/catalog/sense_power/FM89/SC1448/PF253882
- Actuators/output
 - LED Display
 - 16x2 char lines



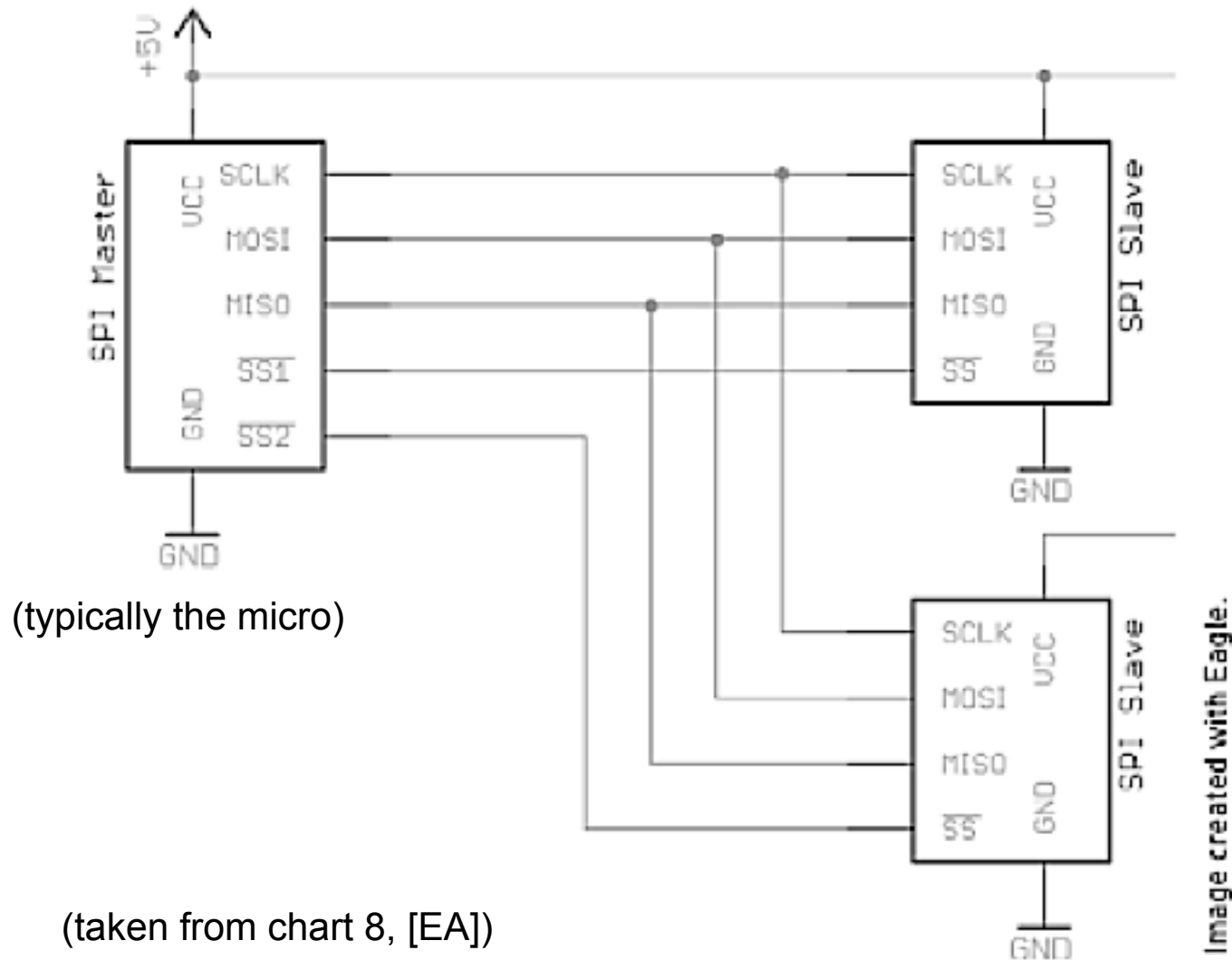
SPI

- Originally introduced by Motorola, the SPI bus implements a *full-duplex* serial protocol that allows for a bidirectional communication between the master and the slaves
 - as I²C it is a serial protocol, based on master-slave schema
 - differently from I²C, SPI uses different lines for transmitting and receiving data, and a specific line to select the slave to communicate with
- It is not an official formal standard
 - so there could be slight differences among implementations and supported commands
- SPI device are synchronous, as I²C
 - data are sent in sync using a shared signal clock (SCLK)

SPI ARCHITECTURE

- Master-slave architecture
 - The SPI bus is controlled by a master device
 - typically the microcontroller
 - it contains one or multiple slave device that receive info from the master
- 3 lines for communicating
 - a shared clock signal (Shared Serial Clock, **SCK**)
 - used to coordinate the communication
 - a shared Master Out Slave In (**MOSI**) line
 - used to send data from the master to the slave
 - a shared Master In Slave Out (**MISO**) line
 - used to send data from the slave to the master
 - a shared SS (**Slave Select**) line
 - used to select the slave

SPI ARCHITECTURE

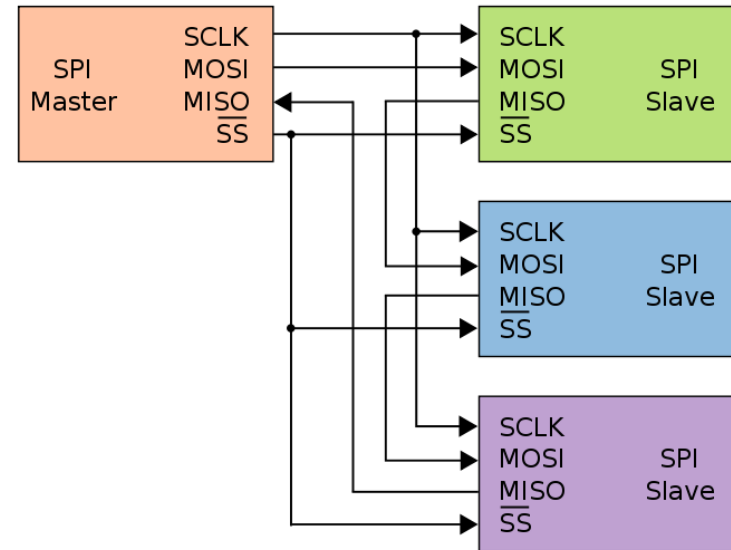
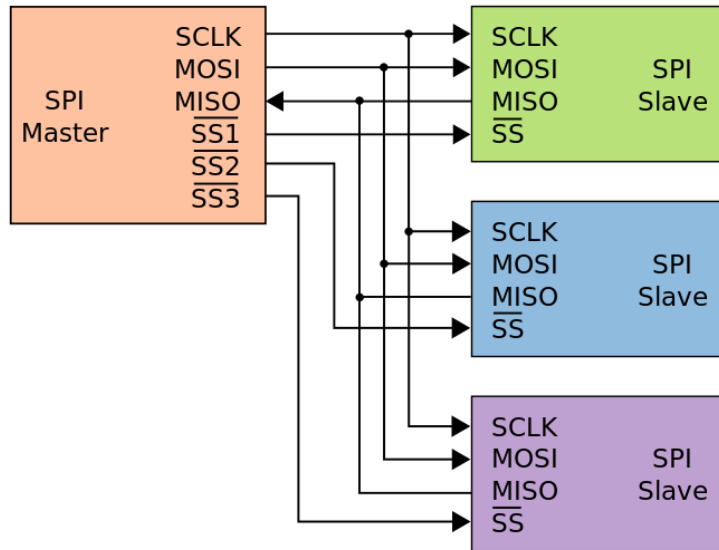


(taken from chart 8, [EA])

Figure 9-1: SPI reference hardware configuration

MULTISLAVE CONNECTIONS

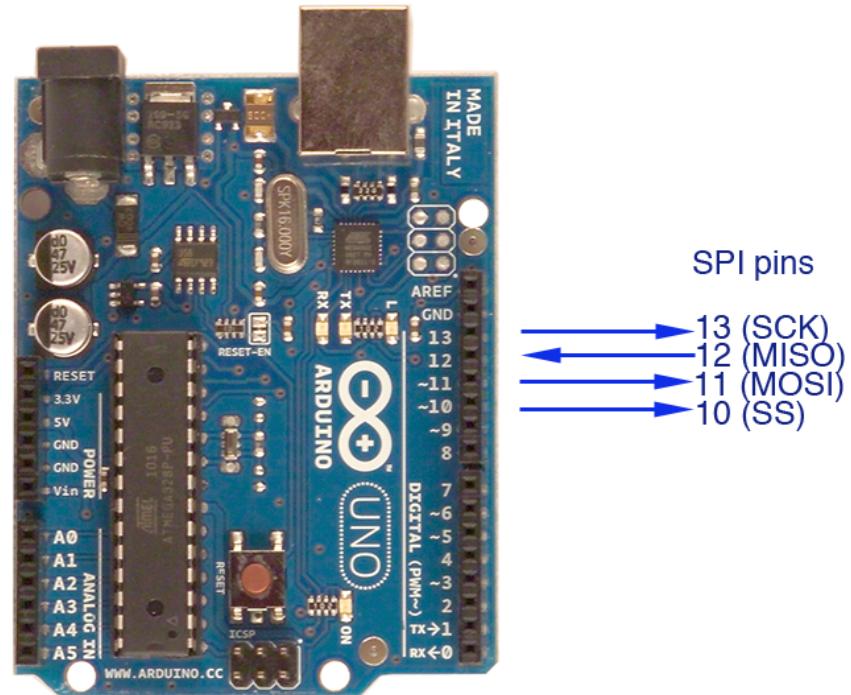
- Two possibilities to connect multiple slaves to the same master
 - using more/multiple SS lines
 - using a *daisy chain* connection



(https://en.wikipedia.org/wiki/Serial_Peripheral_Interface_Bus)

SPI AND ARDUINO/ATmega328P

- Arduino/ATmega328P natively supports SPI
 - 4 GPIO (multiplexed): pin 10, 11, 12, 13
 - 10 => SS
 - 11 => MOSI
 - 12 => MISO
 - 13 => SCK



WIRING: THE SPI LIBRARY

- SPI Library - interact with SPI devices
 - <https://www.arduino.cc/en/Reference/SPI>
- Methods
 - opening and closing the communication session

```
SPI.beginTransaction(settings)
```

```
...
```

```
SPI.endTransaction();
```

- transferring data

```
receivedVal = SPI.transfer(val)
```

```
receivedVal16 = SPI.transfer16(val16)
```

```
SPI.transfer(buffer, size)
```

DETAILS: SETTING

- The configuration specified in `beginTransaction` includes:
 - max speed to be used
 - data format - either MSB (Most Significant Bit) or LSB (Least Significant Bit)
 - sampling data modality
 - 4 transmission modes, specifying the clock edge to be used either for sampling or sending data (clock phase) and the value HIGH or LOW to be used when the clock is idle (clock polarity)

Mode	Clock Polarity (CPOL)	Clock Phase (CPHA)
SPI_MODE0	0	0
SPI_MODE1	0	1
SPI_MODE2	1	0
SPI_MODE3	1	1

- Example:

```
SPI.beginTransaction(SPISettings(14000000, MSBFIRST,  
SPI_MODE0));
```

DETAILS: DATA TRANSFER

- **transfer()** method family to transfer data:

```
receivedVal = SPI.transfer(val)
receivedVal16 = SPI.transfer16(val16)
SPI.transfer(buffer, size)
```

val: byte to be sent on the bus

val16: word to be sent on the bus

buffer: byte array where to send/receive data

- SPI transfer allows for sending and receiving data simultaneously
 - not only single bytes but multiple bytes, using a buffer

PROTOCOL

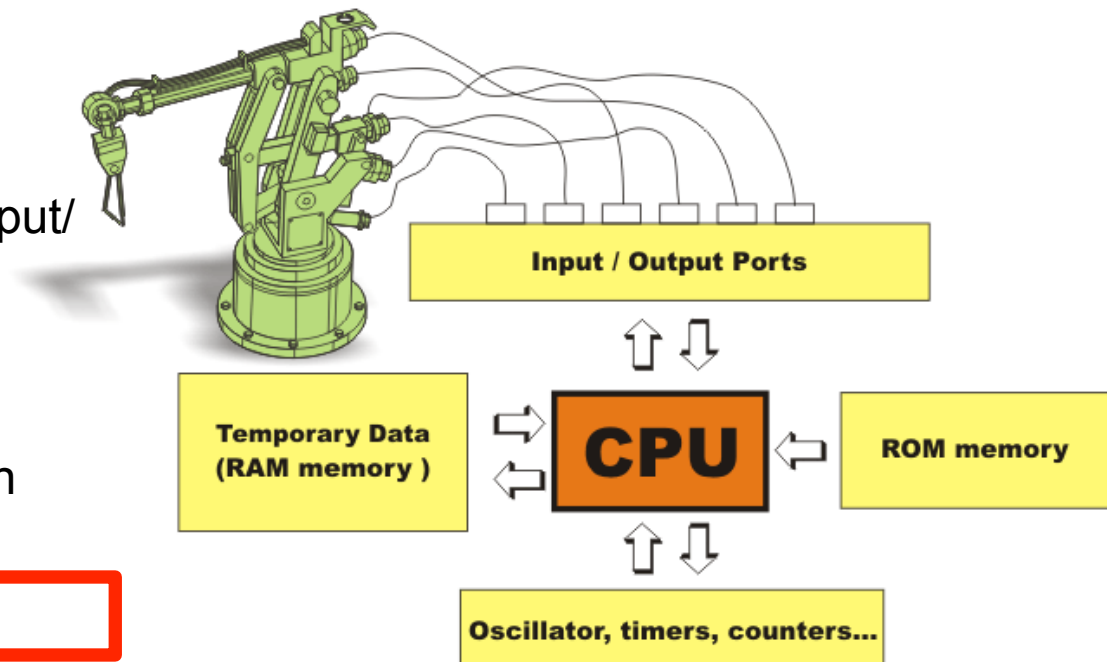
- Sequence of actions to communicate with a slave:
 - we set the slave select pin SS on LOW
 - by means of a digitalWrite on pin SS
 - we call SPI.transfer() to transfer the data
 - we set the slave select pin SS on HIGH

I²C AND SPI - COMPARISON

- Many devices - including accelerometers, digital potentiometers, displays, ... - are available both on I²C and SPI
- Comparison
 - I²C
 - only 2 lines
 - slave addressing modality more agile
 - SPI
 - can work with higher speed
 - pull-up resistors not needed

MICROCONTROLLER ELEMENTS

- CPU
- Memory unit
- Ports for general purpose Input/Output (GPIO)
- Analog/digital converters
- Timers
- Bus for serial communication
- Clock
- **Power circuit**



POWER MANAGEMENT

- A very important functionality supported by most microcontrollers is power management, with the possibility to exploit low power consumption modality (sleep mode)
- ATmega328P Example(*)
 - 5 sleep modalities (increasing power reduction)
 - Idle Mode
 - ADC Noise Reduction Mode
 - Power-save Mode
 - Standby Mode
 - Power-down Mode

(*) http://www.atmel.com/Images/Atmel-2486-8-bit-AVR-microcontroller-ATmega8_L_datasheet.pdf, pag 32

ATMEGA SLEEP MODE

- **Idle Mode**

- stopping the CPU but allowing SPI, USART, Analog Comparator, ADC, Two-wire Serial Interface, Timer/Counters, Watchdog, and the interrupt system to continue operating. This sleep mode basically halts clkCPU and clkFLASH, while allowing the other clocks to run.

- **ADC Noise Reduction Mode**

- stopping the CPU but allowing the ADC, the external interrupts, the Two-wire Serial Interface address watch, Timer/Counter2 and the Watchdog to continue operating (if enabled). This sleep mode basically halts clkI/O, clkCPU, and clkFLASH, while allowing the other clocks to run

- **Power-save Mode**

- This mode is identical to Power-down, with one exception: If Timer/Counter2 is clocked asynchronously, Timer/Counter2 will run during sleep.

- **Standby Mode**

- this mode is identical to Power-down with the exception that the Oscillator is kept running. From Standby mode, the device wakes up in 6 clock cycles

- **Power-down Mode**

- the External Oscillator is stopped, while the external interrupts, the Two-wire Serial Interface address watch, and the Watchdog continue operating (if enabled). Only an External Reset, a Watchdog Reset, a Brown-out Reset, a Two-wire Serial Interface address match interrupt, or an external level interrupt on INT0 or INT1, can wake up the MCU. This sleep mode basically halts all generated clocks, allowing operation of asynchronous modules only.
- When waking up from Power-down mode, there is a delay from the wake-up condition occurs until the wake-up becomes effective. This allows the clock to restart and become stable after having been stopped.

ATMEGA PWR MANAGEMENT

Bit	7	6	5	4	3	2	1	0	
	SE	SM2	SM1	SM0	ISC11	ISC10	ISC01	ISC00	MCUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 13. Sleep Mode Select

SM2	SM1	SM0	Sleep Mode
0	0	0	Idle
0	0	1	ADC Noise Reduction
0	1	0	Power-down
0	1	1	Power-save
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Standby ⁽¹⁾

Note: 1. Standby mode is only available with external crystals or resonators

Table 14. Active Clock Domains and Wake-up Sources in the Different Sleep Modes

Sleep Mode	Active Clock Domains					Oscillators		Wake-up Sources					
	clk _{CPU}	clk _{FLASH}	clk _{IO}	clk _{ADC}	clk _{ASY}	Main Clock Source Enabled	Timer Osc. Enabled	INT1 INT0	TWI Address Match	Timer 2	SPM/EEPROM Ready	ADC	Other I/O
Idle			X	X	X	X	X ⁽²⁾	X	X	X	X	X	X
ADC Noise Reduction				X	X	X	X ⁽²⁾	X ⁽³⁾	X	X	X	X	
Power Down								X ⁽³⁾	X				
Power Save					X ⁽²⁾		X ⁽²⁾	X ⁽³⁾	X	X ⁽²⁾			
Standby ⁽¹⁾						X		X ⁽³⁾	X				

- Notes:
1. External Crystal or resonator selected as clock source
 2. If AS2 bit in ASSR is set
 3. Only level interrupt INT1 and INT0

ARDUINO EXAMPLE

```
void sleepNow()          // here we put the arduino to sleep
{
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);  // sleep mode is set here

    sleep_enable();          // enables the sleep bit in the mcucr register
                             // so sleep is possible. just a safety pin

    /* Now it is time to enable an interrupt.
    attachInterrupt(0,wakeUpNow, LOW); // use interrupt 0 (pin 2) and run function
                                     // wakeUpNow when pin 2 gets LOW

    sleep_mode();            // here the device is actually put to sleep!!
                             // THE PROGRAM CONTINUES FROM HERE AFTER WAKING UP

    sleep_disable();         // first thing after waking from sleep:
                             // disable sleep...
    detachInterrupt(0);      // disables interrupt 0 on pin 2 so the
                             // wakeUpNow code will not be executed
                             // during normal running time.

}
```

CONSUMPTION

(ATMega8 datasheet)

Figure 119. Active Supply Current vs. Frequency (1MHz - 20MHz)

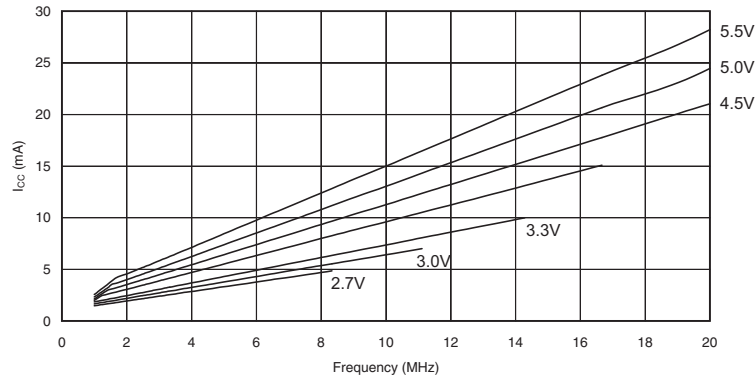
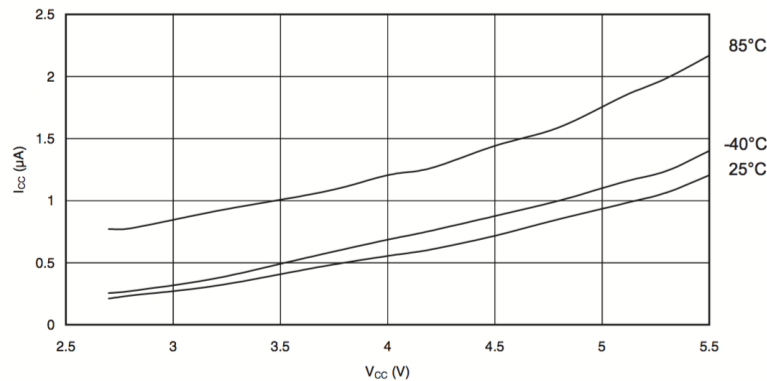


Figure 132. Power-down Supply Current vs. V_{CC} (Watchdog Timer Disabled)



Test examples:

<https://learn.sparkfun.com/tutorials/reducing-arduino-power-consumption>

Figure 126. Idle Supply Current vs. Frequency (1MHz - 20MHz)

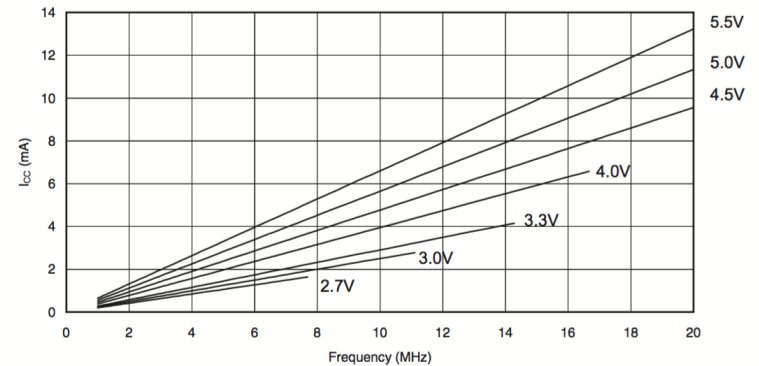


Figure 137. Standby Supply Current vs. V_{CC} (2MHz Resonator, Watchdog Timer Disabled)

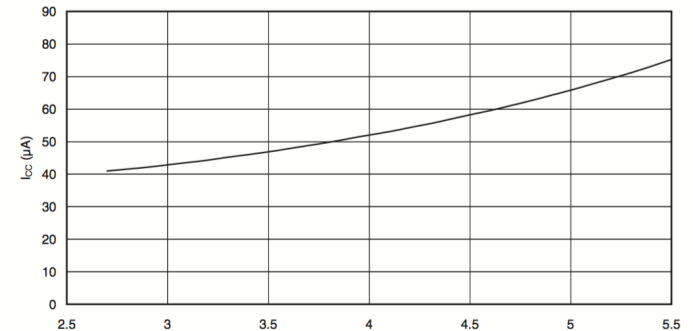
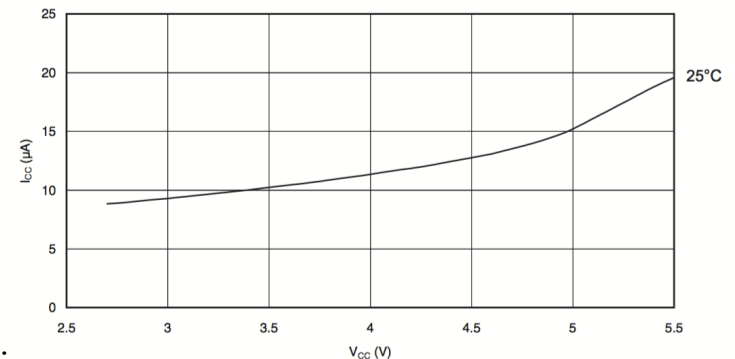


Figure 134. Power-save Supply Current vs. V_{CC} (Watchdog Timer Disabled)



REFERENCES

- **[EA]** Jeremy Blum. *Exploring Arduino*. Wiley
- Wiring
 - <http://wiring.org.co/>
- MikroElektronika books about micro-controllers
 - <http://www.mikroe.com/chapters/view/74/pic-basic-book-chapter-1-world-of-microcontrollers/>